CSE 211 — Hakim Sir

Term-Final question bank with solutions

78 questions from the 2017-18, 2019-20, Jan-2020, 2021-22, 2022-23 and 2023-24 papers, grouped by the practice-tracker tabs and lessons. Each entry has a header line from the tracker (number, exam year, question number, marks, priority, lesson), followed by the question and its worked solution. Where the same question was asked in several papers, the stacked cards share one solution and later entries point back to it.

Priority: High = 2019-20 onward; Medium = the question type repeats in another paper; Low = appears once.

Tab	Questions
1 Intro + Languages	4
2 DFA	13
3 NFA	13
4 Regular Languages	24
5 Pumping Lemma	9
6 CFG	15

Solutions are taken from the CSE 211 Theory of Computation Topic-wise Solved Questions set (credited in that book to Mahdi Hasan, written with Claude Opus 5.5). Check any answer against your lecture notes and the note PDFs in the notes folder before relying on it in the exam.

1 Intro + Languages — Lec01–02 — Introduction & Languages

Notes: CSE211_Hakim_Intro_Languages_Notes.pdf

1. 2023-24 · Q1(a) · 8 marks · ★★★ High · Theory · L1

2023–2024 • Q1(a) Source N01 MARKS 8

What options are available when you confront a problem that appears computationally hard?

SOLUTION

First *establish* the hardness (prove NP-hardness by reducing a known NP-complete problem to it), so you stop searching for an exact fast algorithm. Then:

(i) **Special cases / restricted inputs:** the instances you actually face may be easy (2-SAT, planar or bounded-degree graphs, trees, small numbers ⇒ pseudo-polynomial DP for knapsack).

(ii) **Parameterised algorithms:** exponential only in a small parameter k (e.g. vertex cover in $O(2^k n)$).

(iii) **Approximation algorithms** with a guaranteed ratio (2-approx. vertex cover, Christofides for metric TSP).

(iv) **Heuristics and local search:** greedy, simulated annealing, genetic algorithms — good answers, no guarantee.

(v) **Randomised algorithms** (Monte Carlo, Las Vegas) with good expected behaviour.

(vi) **Smarter exact search:** branch-and-bound, backtracking, SAT/ILP solvers — exponential worst case but often fast in practice.

(vii) **Change/relax the problem** (e.g. LP relaxation) or accept average-case performance.

(viii) **Exploit it:** if a problem is truly hard, it may be useful for security (cryptography).

2. 2022-23 · Q1(a) · 10 marks · ★★★ High · Theory · L1

2022–2023 • Q1(a) Source M49 MARKS 10

Explain the roles of public and private keys in asymmetric cryptography, how they work together to secure communication between two parties, and the role of Theory of Computation in this setting.

SOLUTION

Keys. Each party (say Bob) generates a pair (K_{pub}, K_{priv}). The public key is published; the private key is kept secret. They are mathematically linked (in RSA: $n = pq$, e public, $d = e^{-1} \bmod \varphi(n)$ private), but computing K_{priv} from K_{pub} is believed infeasible.
Confidentiality. Alice encrypts m with Bob's public key: $c = E_{pub}(m)$ ($c = m^e \bmod n$). Only Bob, holding the private key, can decrypt: $m = D_{priv}(c)$ ($c^d \bmod n$). No shared secret had to be exchanged beforehand.
Authentication / integrity. Bob signs with his private key, $\sigma = D_{priv}(h(m))$; anyone verifies with his public key. In practice the slow public-key step is used to agree on a fast symmetric session key (hybrid encryption, TLS).
Role of Theory of Computation. Security rests on *computational complexity*: encryption and decryption with the key are in P (polynomial-time), whereas recovering m or K_{priv} without it requires solving a problem believed intractable (integer factoring, discrete logarithm). These are *one-way (trapdoor) functions* — easy to compute, hard to invert without the trapdoor. Their existence would imply P $\neq$ NP; if P = NP, keys could be found efficiently and public-key cryptography would collapse. Complexity theory gives the language (polynomial time, reductions, average-case hardness) to state and prove such security claims.

3. 2021-22 · Q1(a) · 4 marks · ★★★ High · Theory · L1

2021–2022 • Q1(a) Source M44 MARKS 4

Which of the three traditionally central areas of Theory of Computation is applied in a password-protected computer login system? Explain clearly.

SOLUTION

The three areas are **automata theory**, **computability theory** and **complexity theory**. A login system uses two of them, for two different jobs.

- **Automata theory — checking the password rules.** When a password is chosen, the system checks rules such as "at least 8 characters, contains a digit, contains an uppercase letter, contains no spaces" (the "Password Rules" example of Lecture 1). This needs only *finite memory*: a DFA that remembers $\min(\text{length}, 8)$, whether a digit has been seen and whether an uppercase letter has been seen ($9 \cdot 2 \cdot 2 = 36$ states), plus a dead state entered on a space, reads the password once and accepts exactly the valid ones. So the set of valid passwords is a regular language.
- **Complexity theory — keeping stored passwords secure.** The system stores only a hash $h(\text{password})$, where h is a *one-way function*: computing h is easy (polynomial time), so a login attempt is checked quickly, but inverting h, or guessing the password by brute force over an exponentially large space, is believed computationally infeasible. Security is an argument about the *difficulty* of computation (the same idea as in cryptography), not about what is computable at all.

Computability theory plays no direct role: every task here is decidable; the question is only how much memory or time it needs.

4. 2019-20 · Q1(b) · 15 marks · ★★★ High · Theory · L1

2019–2020 • Q1(b) Source M40 MARKS 15

Describe a scenario that specifically requires computationally hard problems, even though easy problems are generally preferable.

SOLUTION

Secure online banking / e-commerce. A customer sends a card number to a bank over the open Internet, where an eavesdropper can read every packet. The bank publishes an RSA public key (n, e) with $n = pq$ a product of two secret 1024-bit primes; the browser encrypts with it, and only the bank (knowing p, q, hence d) decrypts.

- The legitimate parties need *easy* problems: modular exponentiation is polynomial time.
- The attacker must be faced with a *hard* problem: factoring n (or computing e-th roots mod n). The best known algorithms take super-polynomial time, so even all the world's computers would need astronomically long.

If factoring were easy, anyone could read everyone's traffic. Here hardness is a feature, not an obstacle: the *gap* between easy (with the trapdoor) and hard (without it) is what gives security. Other examples: password hashing, digital signatures, proof-of-work in cryptocurrencies, CAPTCHAs.

2 DFA — Lec03 — Deterministic Finite Automata

Notes: CSE211_Hakim_DFA_Notes.pdf

5. 2023-24 · Q2(c) · 5 marks · ★★★ High · Theory · L3

2023–2024 • Q2(c) Source N01 — MARKS 5

Does the number of accepting DFA states affect the numbers of rows or columns in its transition table? Explain.

SOLUTION

No. A transition table of a DFA $M = (Q, \Sigma, \delta, q_0, F)$ is simply a tabulation of the total function $\delta : Q \times \Sigma \to Q$.

- There is exactly **one row per state** $q \in Q$, so the number of rows is $|Q|$, regardless of how many of those states are in F.
- There is exactly **one column per input symbol** $a \in \Sigma$, so the number of columns is $|\Sigma|$.

Accepting states are only *marked* (with $*$) in the row label; changing F changes the markings, never the shape. For example, the parity DFA below has a 2×2 table whether $F = \{E\}$ (even number of 1's) or $F = \{O\}$ (odd) or $F = \{E, O\}$ (everything):

δ	0	1
$\to *E$	E	O
O	O	E

Answer. Rows $= |Q|$ and columns $= |\Sigma|$; the size of F only affects which rows carry a $*$.

6. 2019-20 · Q2(a) · 5 marks · ★★★ High · Theory · L3

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q2(a) Source M40 — MARKS 5

Briefly explain the role of a state in a finite automaton.

2007–2008 • Q2(c) Source M04 — MARKS $5\frac{1}{3}$

Describe the most important role played by states in an automaton. Can this role have infinite capacity? Why?

SOLUTION

SCOPE *2019–2020 Q2(a) asks only for the role of a state (first paragraph).*

Role: states are the machine's only memory. A finite automaton has no tape or stack; after reading a prefix x it is in one state, and that state must summarise *everything about x that matters for the future*. In short, **a state is a compressed summary of the past**: it captures exactly the information needed to determine future behaviour, ignoring irrelevant history. Two prefixes may therefore lead to the same state only if no continuation can ever tell them apart. Example: for "even number of 1's" it suffices to remember the parity, so two states E, O suffice; for "ends in 01" we remember the relevant suffix (ε, 0, 01).

Infinite capacity? No. A finite automaton has finitely many states $|Q| = k$, so it can remember at most $\lceil \log_2 k \rceil$ bits about an arbitrarily long input. It cannot, for instance, count unboundedly: to recognise $\{0^n 1^n\}$ it would have to distinguish $0^1, 0^2, 0^3, \ldots$, i.e. infinitely many states. If we allowed infinitely many states the device would no longer be a finite automaton — in fact it could accept *any* language (one state per prefix, a "trie"), and it could not be built. Unbounded memory is exactly what PDAs (a stack) and Turing machines (a tape) add.

Answer. The state is a finite memory summarising the relevant past; its capacity is necessarily finite, which is why FAs recognise only regular languages.

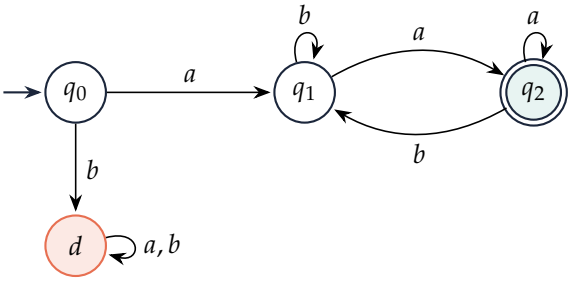
7. 2023-24 · Q2(b) · 15 marks · ★★★ High · Design · L4

2023–2024 • Q2(b) Source N01 **MARKS 15**

Show that $L = \{awa \mid w \in \{a, b\}^*\}$ is regular by constructing a DFA.

SOLUTION

L = strings of length ≥ 2 that begin and end with a. After the first a, state q_2 means "last symbol is a" (accept) and q_1 means "last symbol is b or only the first a was read".



Note that the single string a ends in q_1 and is rejected, as required ($|awa| \geq 2$). Since a DFA recognises L, L is regular.

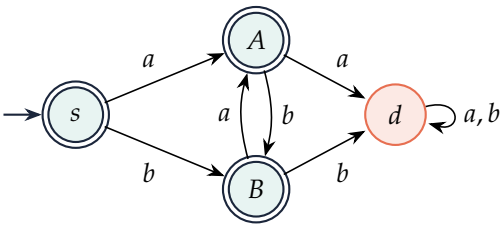
8. 2022-23 · Q1(c) · 13 marks · ★★★ High · Design · L4

2022–2023 • Q1(c) Source M49 **MARKS 13**

Construct a DFA for $\{w \in \{a,b\}^* \mid \text{neither } aa \text{ nor } bb \text{ is a substring}\}$. A transition-table representation is required.

SOLUTION

The strings are exactly the alternating strings. Remember the last symbol.



δ	a	b
$\to *s$	A	B
$*A$	d	B
$*B$	A	d
d	d	d

s: nothing read; A/B: last symbol a/b; d: dead.

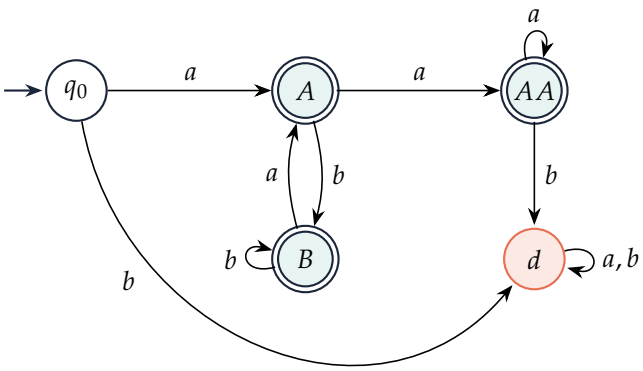
9. 2021-22 · Q1(b) · 10 marks · ★★★ High · Design · L4

2021–2022 • Q1(b) Source M44 **MARKS 10**

Design a DFA with no more than five states over $\{a,b\}$, accepting strings that begin with a and do not contain aab. Give only the state diagram.

SOLUTION

States: q_0 start; A = last symbol a (single); AA = ends in aa (a following b would complete aab); B = last symbol b; d = dead. Five states.



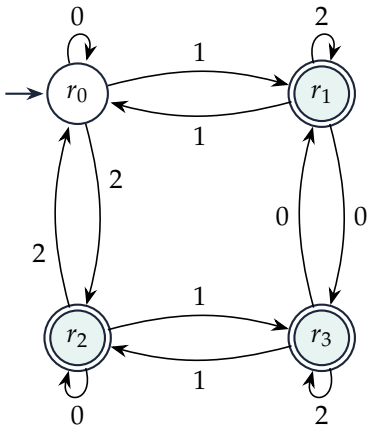
10. 2019-20 · Q2(c) · 15 marks · ★★★ High · Design · L4

2019–2020 • Q2(c) Source M40 — **MARKS 15**

Draw a DFA accepting base-3 numbers (digits 0, 1, 2) that are not divisible by 4.

SOLUTION

Reading digit d after a prefix with value v gives value $3v + d$, so with $r = v \bmod 4$: $r \mapsto (3r + d) \bmod 4$. The start state is $r = 0$ (empty prefix, value 0 — non-accepting, so ε is rejected). Accept $r \in \{1, 2, 3\}$.



r	0	1	2
$\to r_0$	r_0	r_1	r_2
$*r_1$	r_3	r_0	r_1
$*r_2$	r_2	r_3	r_0
$*r_3$	r_1	r_2	r_3

e.g. $12_3 = 5$: $r_0 \xrightarrow{1} r_1 \xrightarrow{2} r_1$ accept ($5 \bmod 4 = 1$).

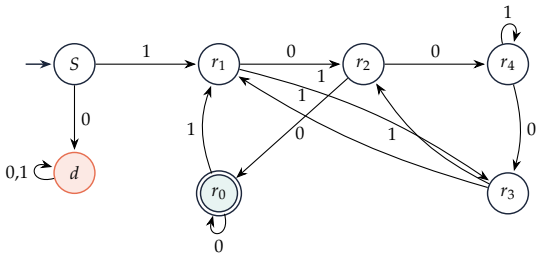
11. 2017-18 · Q2(b) · 17 marks · ★★ Medium · Design · L4

2017–2018 • Q2(b) Source M32 — **MARKS 10 + 7 = 17**

Draw DFA state diagrams over $\{0, 1\}$ for (i) strings beginning with 1 whose binary value is divisible by 5 (accept 101, 1010, 1111; reject 0, 100, 0101, 111); (ii) strings beginning with 1 whose binary value is a power of 2.

SOLUTION

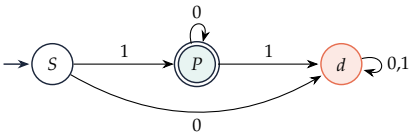
(i) Appending bit b maps value v to $2v + b$, so the remainder $r = v \bmod 5$ updates as $r \mapsto (2r + b) \bmod 5$. A start state S forces the first bit to be 1.



	0	1
$\to S$	d	r_1
$*r_0$	r_0	r_1
r_1	r_2	r_3
r_2	r_4	r_0
r_3	r_1	r_2
r_4	r_3	r_4

Checks: 1111: $r_1 \to r_3 \to r_2 \to r_0$ (15) accept; 111: $r_1 \to r_3 \to r_2$ (7) reject; 0101: $S \xrightarrow{0} d$ reject.

(ii) Powers of 2 starting with 1 are exactly 10^*.



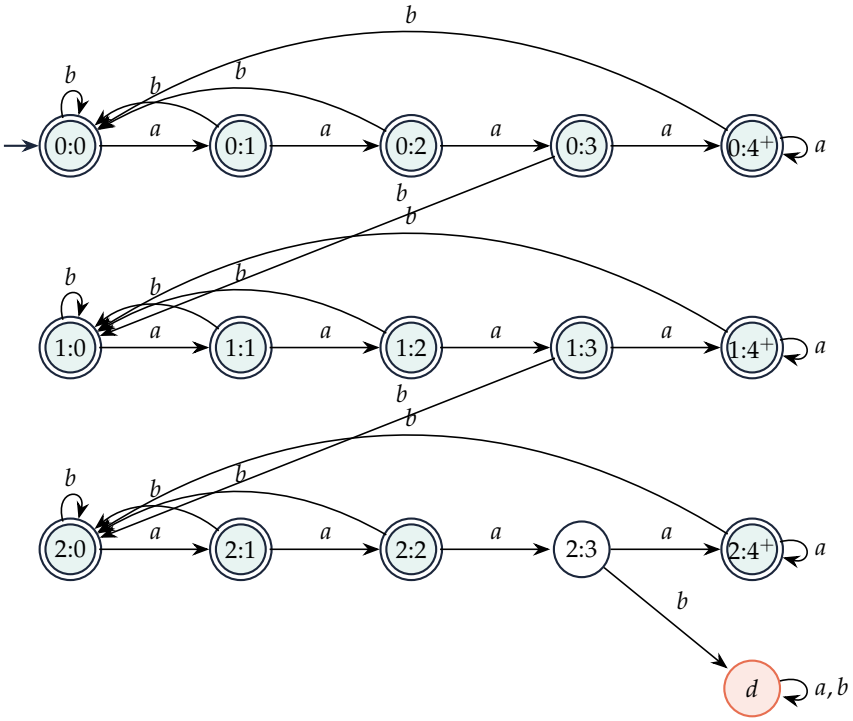
12. 2023-24 · Q1(b) · 17 marks · ★★★ High · Design · L5

2023–2024 • Q1(b) Source N01 **MARKS 17**

A run is a maximal substring of length at least two consisting of one repeated symbol. For example, *abbbaab* has a b-run of length 3 and an a-run of length 2. Design a DFA over $\{a, b\}$ for strings with at most two runs of a's of length exactly three. Give only the state diagram.

SOLUTION

Idea. Remember two things: c = how many completed a-runs of length exactly 3 have been seen $(0, 1, 2)$, and ℓ = length of the current block of a's $(0, 1, 2, 3$ or "4^+"$)$. On a: $\ell \mapsto \min(\ell + 1, 4)$. On b: the block ends; if $\ell = 3$ then c increases (a third one sends us to the dead state d), and $\ell \mapsto 0$. At the end of input a block of length exactly 3 that is still open also counts, so state $(2{:}3)$ is *rejecting* while every other non-dead state accepts. State $c{:}\ell$ is drawn in row c, column ℓ.



All 15 states $c{:}\ell$ except 2:3 are accepting. (Brute-force check: this DFA agrees with the definition on all $2^{13} - 1$ strings of length ≤ 12.)

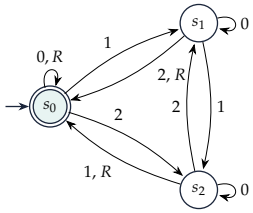
13. 2022-23 · Q1(b) · 12 marks · ★★★ High · Theory · L5

2022–2023 • Q1(b) Source M49 **MARKS 12**

An automaton over $\{R, 0, 1, 2\}$ keeps the numerical-symbol sum modulo 3, resets it to 0 on R, and accepts remainder 0. A three-state design tracks all remainders. Null Pointer proposes using two states, one for remainder 0 and one for a nonzero remainder. Analyze this claim in detail.

SOLUTION

The correct design. States s_0, s_1, s_2 remember the current sum mod 3; on digit d go from s_i to $s_{(i+d) \bmod 3}$, on R go to s_0; accept in s_0.



Why two states cannot work. Suppose a DFA with states Z (remainder 0) and N ("nonzero") existed. After reading 1 it is in N, and after reading 2 it is also in N. Now append 1:

$$1 \cdot 1 = 11 :\ 2 \not\equiv 0 \text{ (reject)}, \qquad 2 \cdot 1 = 21 :\ 3 \equiv 0 \text{ (accept)}.$$

A DFA is deterministic, so from the *same* state N the same suffix 1 must lead to the same state and the same verdict — contradiction. The state N has lost the information "is the remainder 1 or 2?", which is exactly what is needed to decide the future.

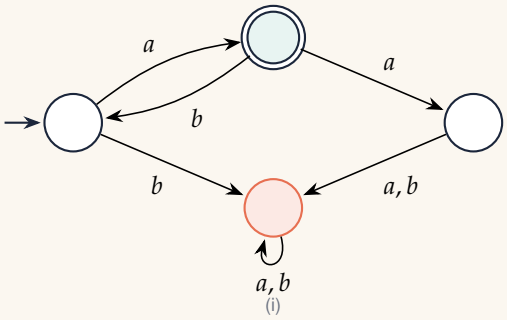
Lower bound (distinguishable prefixes). The prefixes ε, 1, 2 are pairwise distinguishable: suffix 2 separates 1 (sum 3, accept) from ε and 2 (sums 2 and 4, reject); suffix 1 separates 2 (sum 3, accept) from ε (sum 1, reject). If two of them led to the same state, appending the separating suffix would force the same verdict for both, which is impossible. Hence *every* DFA for the language needs at least 3 states.

Answer. The claim is wrong: "nonzero" merges remainders 1 and 2, which react differently to future input (e.g. suffix 1). Three states are necessary and sufficient.

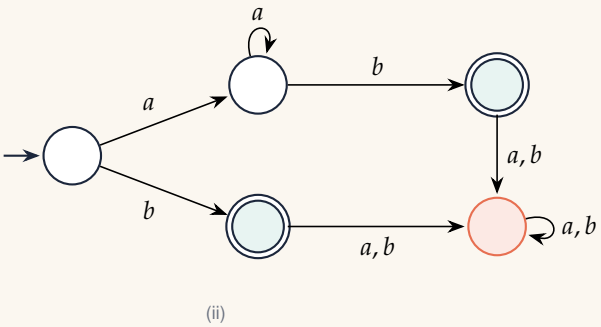
14. 2022-23 · Q2(a) · 20 marks · ★★★ High · Theory · L5

2022–2023 • Q2(a)(i–v) Source M49–M50 **MARKS 20 (shared)**

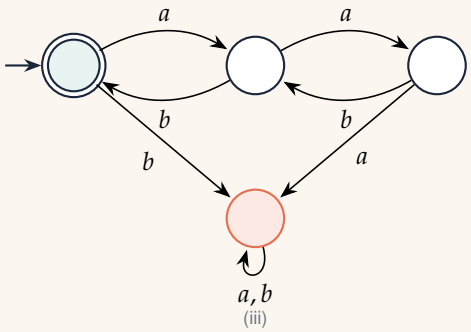
Describe informally the language accepted by each of the five shown automata.



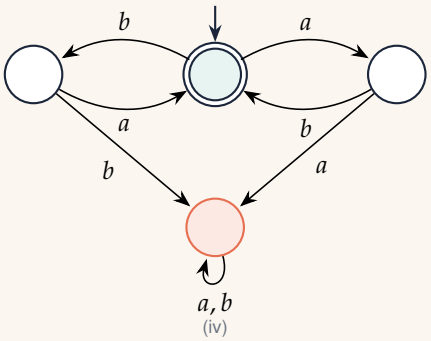
(i)



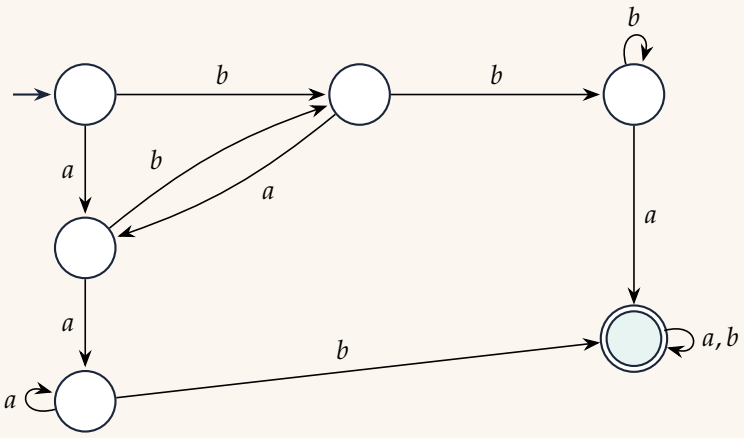
(ii)



(iii)



(iv)



(v)

SOLUTION

Trace what each state "remembers".

(i) From the start, a leads to the only accepting state; from there b returns to the start and a falls (via an extra state) into the trap. Accepted strings are therefore a, aba, $ababa$, ...

Answer. $L_1 = (ab)^*a$: strings that alternate a, b, starting and ending with a.

(ii) The upper branch reads a^+ then a single b to accept; the lower branch accepts the single symbol b. Any further symbol goes to the trap.

Answer. $L_2 = a^*b = \{a^n b \mid n \geq 0\}$.

(iii) Think of a as "(" and b as ")". The three non-trap states record the current depth $0, 1, 2$; a third unmatched a (depth 3) or a b at depth 0 goes to the trap, and we accept at depth 0.

Answer. $L_3 = (a(ab)^*b)^*$: strings with equally many a's and b's in which every prefix satisfies $0 \leq \#_a - \#_b \leq 2$ (balanced "parentheses" of nesting depth at most 2).

(iv) From the centre (accepting) state the machine must read either ab or ba to return; aa or bb inside a block leads to the trap.

Answer. $L_4 = (ab \cup ba)^*$: even-length strings in which positions $2i-1, 2i$ always hold different symbols.

(v) The left column remembers "last symbol a"; the lower-left state remembers "ends in aa" (it loops on a); a b there completes aab. Symmetrically the top-right state remembers "ends in bb" and an a completes bba. The final state is a sink.

Answer. $L_5 = \{w \mid w$ contains aab or bba as a substring$\}$.

15. 2022-23 · Q3(a) · 15 marks · ★★★ High · Design · L5

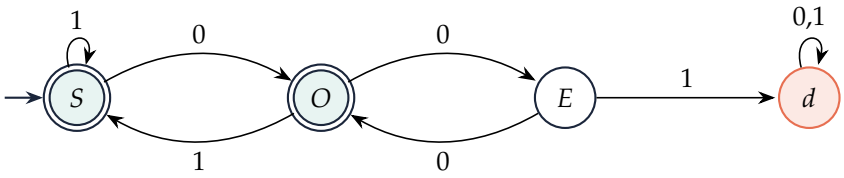
2022–2023 • Q3(a) Source M51 **MARKS 15**

Construct a DFA accepting binary strings in which no run of 0's has even length ≥ 2. A run is a maximal block of consecutive 0's: 00 has one run of length 2; 000010 has runs of lengths 4 and 1. Reject 0001100, accept 01110001 and 11111. Give a state diagram and explain the design idea.

SOLUTION

Idea. Every maximal 0-run must have *odd* length. While inside a run we only need its parity; the verdict on a run is given when it ends — either by a 1 or by the end of the input.

- S: not inside a 0-run (start, or last symbol 1) — accept.
- O: inside a 0-run of odd length so far — accept (if the input ends here the run is odd).
- E: inside a 0-run of even length so far — reject at end; a 1 here closes an even run $\Rightarrow$ dead d.

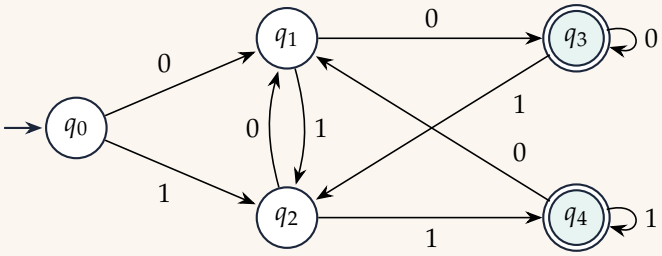


Checks: 0001100: runs 000 (odd), 00 (even, ends in E) $\Rightarrow$ reject. 01110001: $S \xrightarrow{0} O \xrightarrow{111} S \xrightarrow{000} O \xrightarrow{1} S$ accept. 11111: stays in S, accept.

16. 2020 (Jan term) · Q2(a) · 7 marks · ★★★ High · Theory · L5

January 2020 Term • Q2(a) Source M36 **MARKS 7**

State informally the language accepted by the shown DFA.



SOLUTION

q_1 means "the last symbol is 0 but the last two are not 00"; q_3 means "the last two symbols are 00". Symmetrically q_2/q_4 for 1. Every transition on 0 goes to q_1 or q_3, every transition on 1 to q_2 or q_4, and we accept exactly in q_3, q_4.

Answer. $L = \{w \in \{0,1\}^* \mid w \text{ ends in } 00 \text{ or } 11\} = (0 \cup 1)^*(00 \cup 11)$, i.e. the last two symbols are equal.

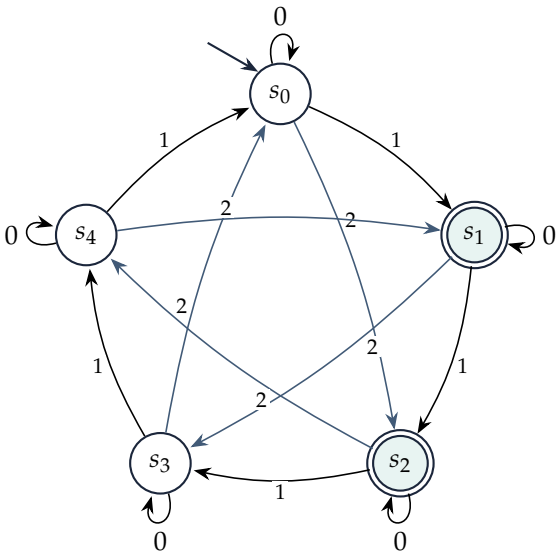
17. 2019-20 · Q1(c) · 15 marks · ★★★ High · Design · L5

2019–2020 • Q1(c) Source M40 **MARKS 15**

Draw a DFA over $\{0, 1, 2, R\}$ keeping the sum of numerical input symbols, resetting it to 0 on R. Accept if the sum is 1 or 2 modulo 5. A state diagram suffices.

SOLUTION

Only the sum mod 5 matters, so use states $s_0, \ldots, s_4$; on digit d: $s_i \to s_{(i+d) \bmod 5}$; on R: $\to s_0$. Accept s_1, s_2.



In addition, **every** state has an R-edge to s_0 (omitted from the drawing for clarity; s_0 also loops on R). Transition table:

	0	1	2	R
$\rightarrow s_0$	s_0	s_1	s_2	s_0
$*s_1$	s_1	s_2	s_3	s_0
$*s_2$	s_2	s_3	s_4	s_0
s_3	s_3	s_4	s_0	s_0
s_4	s_4	s_0	s_1	s_0

3 NFA — Lec04 — Nondeterminism and NFAs

Notes: CSE211_Hakim_NFA_Notes.pdf

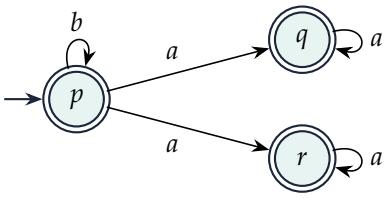
18. 2023-24 · Q3(a) · 15 marks · ★★★ High · Design · L7

2023–2024 • Q3(a) Source N02 **MARKS 15**

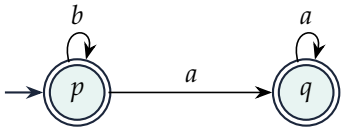
Find a three-state NFA for $L = \{a^n \mid n \geq 1\} \cup \{b^m a^k \mid m, k \geq 0\}$. Give only the diagram. Can an NFA with fewer than three states accept it? Elaborate.

SOLUTION

Three-state NFA (one branch per part of the union, sharing the start):



$p \xrightarrow{a} r \xrightarrow{a} \cdots$ gives a^n $(n \geq 1)$; $p \xrightarrow{b^m} p \xrightarrow{a} q \xrightarrow{a^{k-1}} q$ gives $b^m a^k$.
Fewer states? Yes — *two* suffice. Observe $\{a^n \mid n \geq 1\} \subseteq \{b^m a^k\}$ (take $m = 0$), so $L = b^* a^*$, and state r above is redundant:



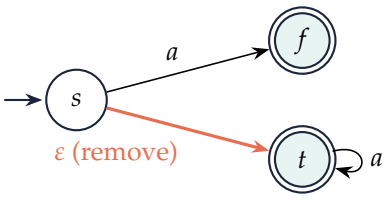
One state is impossible. With one state s: since $\varepsilon \in L$, s must be accepting, so $L(N) = S^*$ where S is the set of symbols on its self-loops. $S = \{a, b\}$ accepts $ab \notin L$; $S = \{a\}$ misses b; $S = \{b\}$ misses a; $S = \emptyset$ gives $\{\varepsilon\}$. Hence the minimum is **2 states**.

19. 2023-24 · Q4(a) · 10 marks · ★★★ High · Design · L7

2023–2024 • Q4(a) Source N02 **MARKS 10**

Find an NFA accepting $\{a\}^*$ such that removing just one edge, with no other changes, makes it accept $\{a\}$. Give only the state diagram.

SOLUTION



With the coral ε-edge, $L = \{a\} \cup a^* = a^*$. Removing it leaves only $s \xrightarrow{a} f$, so $L = \{a\}$. (The removable edge must be an ε-edge: $\varepsilon \in a^*$ but $\varepsilon \notin \{a\}$, and an accepting path for ε uses only ε-edges.)

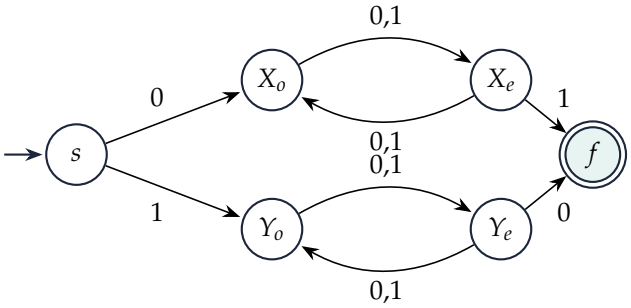
20. 2022-23 · Q4(b) · 15 marks · ★★★ High · Design · L7

2022–2023 • Q4(b) Source M51 MARKS 15

Draw an NFA for binary strings beginning and ending with different symbols and having odd length. A state diagram suffices.

SOLUTION

Branch on the first symbol, track length parity, and guess that the next symbol is the last one — allowed only when it differs from the first symbol and makes the length odd (i.e. it is read from an even-length state).



X: began with 0, Y: began with 1; subscript = parity of the length read so far. Example: 001: $s \to X_o \to X_e \to f$ ✓; 01 (even length) cannot reach f.

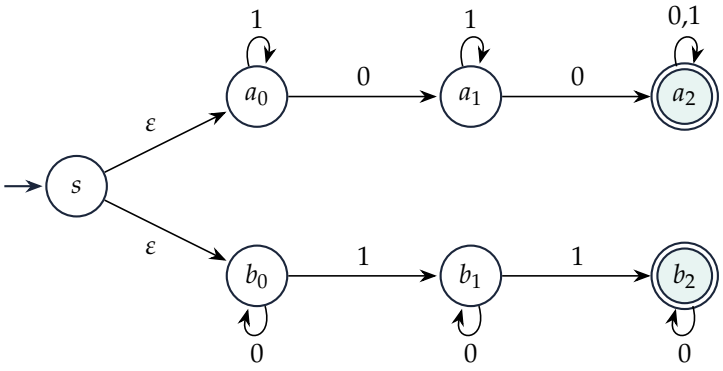
21. 2021-22 · Q3(b) · 10 marks · ★★★ High · Design · L7

2021–2022 • Q3(b) Source M45 MARKS 10

Draw an NFA accepting binary strings containing at least two 0's or exactly two 1's. A state diagram suffices.

SOLUTION

Union of two simple machines via a new start state with ε-edges.



Upper branch counts 0's up to 2; lower branch counts 1's and has no move on a third 1 (that branch dies).

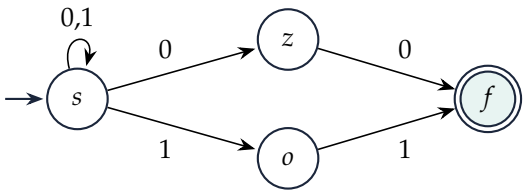
22. 2020 (Jan term) · Q2(b) · 13 marks · ★★★ High · Design · L7

January 2020 Term • Q2(b) Source M36 **MARKS 13**

Draw an equivalent NFA for the DFA in 2(a) (the five-state DFA accepting strings that end in 00 or 11; see Topic 01).

SOLUTION

The DFA accepts $(0 \cup 1)^*(00 \cup 11)$. An NFA simply guesses where the final pair starts:



4 states instead of 5, with one nondeterministic choice at s for each symbol.

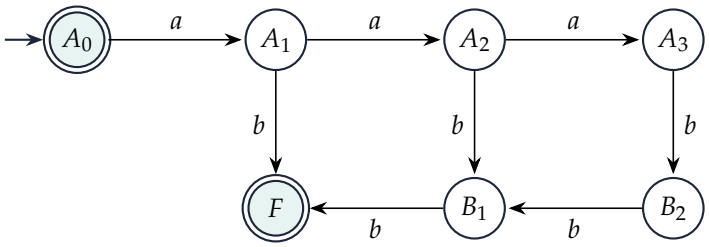
23. 2019-20 · Q3(b) · 15 marks · ★★★ High · Design · L7

2019–2020 • Q3(b) Source M40 **MARKS 15**

Design an NFA for $L = \{a^n b^n \mid n \leq 3\}$ over $\{a, b\}$. Give only a state diagram. Using more than the required minimum states may be penalized.

SOLUTION

$L = \{\varepsilon, ab, aabb, aaabbb\}$. After k a's jump to the point of the b-chain that still needs k b's:



Fewer than 7 states is impossible. Let N be any NFA with $L(N) = L$. Since $a^3b^3 = s_1 s_2 \cdots s_6 \in L$, N has an accepting computation on it; let r_i be the state it is in after reading the first i symbols ($i = 0, 1, \ldots, 6$). Suppose $r_i = r_j$ for some $i < j$. Then the part of the computation between these two moments reads the $j - i \geq 1$ symbols $s_{i+1} \cdots s_j$ and returns to the same state, so it can be traversed twice: N also accepts $s_1 \cdots s_i \, (s_{i+1} \cdots s_j)^2 \, s_{j+1} \cdots s_6$, a string of length $6 + (j - i) > 6$. But every string of L has length at most 6 — contradiction. Hence $r_0, \ldots, r_6$ are 7 different states, and every NFA for L has at least 7 states; the NFA above is optimal.

24. 2017-18 · Q3(a) · 18 marks · ★★ Medium · Design · L7

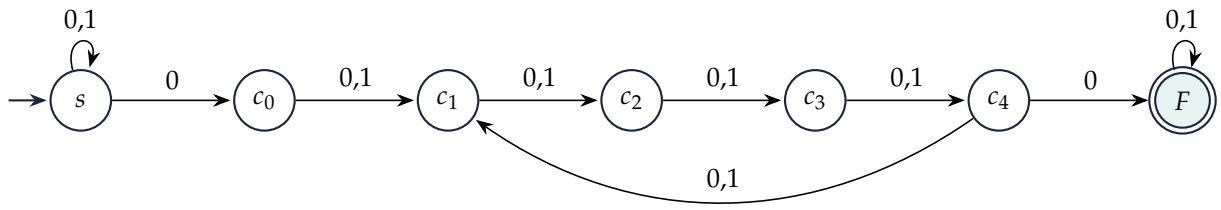
2017–2018 • Q3(a) Source M32 **MARKS 18**

Design an NFA for binary strings containing two 0's with a nonzero multiple of 4 characters between them; other 0's may occur. Give a state diagram and transition table. Source accepted examples: 00000, 101111010, 01000010111011. Source rejected examples: 00, 10101010.

SOLUTION

> **NOTE** **With the stated condition, 00000 should be *rejected*: any two of its 0's have $0, 1, 2$ or 3 characters between them, never a nonzero multiple of 4. The NFA below follows the stated condition; the other four examples are handled correctly.**

Guess the first of the two 0's, then count the characters after it modulo 4 (at least 4 of them), and guess that a 0 read when the count is $\equiv 0$ (and ≥ 4) is the second one.



State	0	1
$\to s$	$\{s, c_0\}$	$\{s\}$
c_0	$\{c_1\}$	$\{c_1\}$
c_1	$\{c_2\}$	$\{c_2\}$
c_2	$\{c_3\}$	$\{c_3\}$
c_3	$\{c_4\}$	$\{c_4\}$
c_4	$\{F, c_1\}$	$\{c_1\}$
$*F$	$\{F\}$	$\{F\}$

c_k: k characters read since the guessed first 0 (c_4 stands for any count $\equiv 0 \pmod 4$, ≥ 4). Example 101111010: 0 at position 2 $\to c_0$; 1111 $\to c_4$; next 0 $\to F$ ✓. 10101010: gaps are 1, 3, 5 characters, never $\equiv 0$, so F is unreachable.

25. 2021-22 · Q2(a) · 4 marks · ★★★ High · Theory · L9

2021–2022 • Q2(a) Source M45 **MARKS 4**

DFA and NFA recognize the same class of languages. Why is this equivalence both surprising and useful?

SOLUTION

Surprising. An NFA may branch into many simultaneous computations, use ε-moves and "guess" (e.g. where a suffix begins). Intuitively this looks strictly more powerful than a single deterministic thread — for Turing-machine time complexity the analogous question (P vs NP) is still open, and for PDAs nondeterminism *does* add power. Yet for finite automata the whole set of currently-active NFA states is itself a finite object (a subset of Q), so a DFA can track it: the subset construction removes nondeterminism at the cost of at most $2^{|Q|}$ states.

Useful.

- *Design*: NFAs are much easier to build and smaller (e.g. "the 3rd symbol from the end is 1" needs 4 NFA states but 8 DFA states).
- *Proofs*: closure under union, concatenation and star, and the conversion regex $\to$ automaton, are trivial with NFAs; the equivalence transfers them to DFAs.
- *Implementation*: DFAs run in one table lookup per symbol, can be complemented by swapping F, and tested for equivalence. We design with NFAs and execute with DFAs.

26. 2021-22 · Q3(c) · 9 marks · ★★★ High · Trace · L9

2021–2022 • Q3(c) Source M45–M46 **MARKS 9**

Convert the given NFA into a DFA. Only the DFA state diagram is required.

State	a	b	ε
q_1	$\{q_3\}$	$\emptyset$	$\{q_2\}$
q_2	$\{q_1\}$	$\emptyset$	$\emptyset$
q_3	$\{q_2\}$	$\{q_2, q_3\}$	$\emptyset$

SOLUTION

NOTE **The table marks neither start nor accepting states. This is the NFA of Sipser Exercise 1.16(b), whose start state is q_1 and accepting state is q_2; we use that.**

Step 1: remove the ε-transitions

$\delta'(q,x) = E\big(\bigcup_{r\in E(q)} \delta(r,x)\big)$ and $F' = \{q \mid E(q) \cap F \neq \emptyset\}$ with $F = \{q_2\}$:

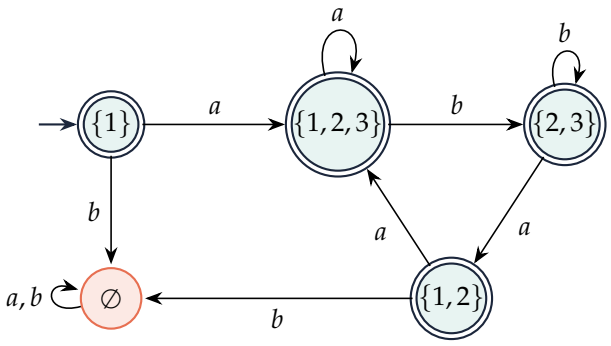
q	$E(q)$	$\delta'(q,a)$	$\delta'(q,b)$
$*q_1$	$\{q_1, q_2\}$	$E(\{q_3, q_1\}) = \{q_1, q_2, q_3\}$	$\emptyset$
$*q_2$	$\{q_2\}$	$E(\{q_1\}) = \{q_1, q_2\}$	$\emptyset$
q_3	$\{q_3\}$	$E(\{q_2\}) = \{q_2\}$	$E(\{q_2, q_3\}) = \{q_2, q_3\}$

$F' = \{q_1, q_2\}$ (q_1 reaches q_2 by an ε-move).

Step 2: subset construction from $\{q_1\}$

$\delta_D(R,x) = \bigcup_{r\in R} \delta'(r,x)$; a subset is accepting iff it meets F'. Only reachable subsets are built:

DFA state	a	b
$\to *\{q_1\}$	$\{q_1, q_2, q_3\}$	$\emptyset$
$*\{q_1, q_2, q_3\}$	$\{q_1, q_2, q_3\}$	$\emptyset \cup \emptyset \cup \{q_2, q_3\} = \{q_2, q_3\}$
$*\{q_2, q_3\}$	$\{q_1, q_2\} \cup \{q_2\} = \{q_1, q_2\}$	$\{q_2, q_3\}$
$*\{q_1, q_2\}$	$\{q_1, q_2, q_3\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$



Five reachable states (numbers abbreviate q_i); all except $\emptyset$ are accepting.

27. 2020 (Jan term) · Q1(c) · 12 marks · ★★★ High · Trace · L9

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

January 2020 Term • Q1(c) Source M36 — **MARKS 12**

For the given NFA, give general forms of all accepted strings of length at most 3, with explanations.

State	a	b	c	ε
$\to p$	$\{p\}$	$\{q\}$	$\{r\}$	$\emptyset$
q	$\{q\}$	$\{r\}$	$\emptyset$	$\{p\}$
$*r$	$\{r\}$	$\emptyset$	$\{p\}$	$\{q\}$

2017–2018 • Q4(a) Source M33 — **MARKS 15**

For the given NFA (same table as above): (i) give, with explanations, general forms of every accepted string of length at most 3; (ii) convert it to a DFA (detailed computations optional).

2008–2009 • Q6(c) Source M07 — **MARKS $5\frac{1}{3}$**

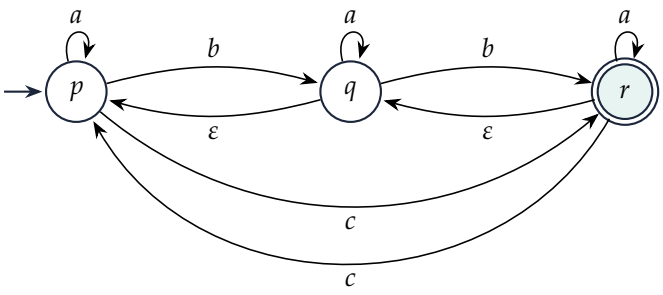
For the ε-NFA in the table: (i) compute the ε-closure of each state; (ii) convert it to a DFA.

State	ε	a	b	c
$\to p$	$\emptyset$	$\{p\}$	$\{q\}$	$\{r\}$
q	$\{p\}$	$\{q\}$	$\{r\}$	$\emptyset$
$*r$	$\{q\}$	$\{r\}$	$\emptyset$	$\{p\}$

SOLUTION

SCOPE *January 2020 Term Q1(c) asks only for the accepted strings of length ≤ 3 (last part); 2008–2009 Q6(c) asks only for the ε-closures and the DFA (Steps 1–2); 2017–2018 Q4(a) asks for the strings of length ≤ 3 and the DFA.*

All three tables describe the *same* ε-NFA (only the column order differs).



Step 1: ε-closures and removal of the ε-transitions

$E(p) = \{p\}$, $E(q) = \{q, p\}$, $E(r) = \{r, q, p\}$. With $\delta'(s, x) = E\big(\bigcup_{t \in E(s)} \delta(t, x)\big)$ and $F' = \{s \mid E(s) \cap F \neq \emptyset\}$:

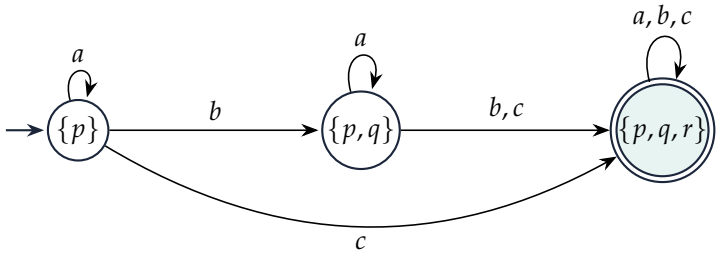
	a	b	c
$\to p$	$E(\{p\}) = \{p\}$	$E(\{q\}) = \{p,q\}$	$E(\{r\}) = \{p,q,r\}$
q	$E(\{p,q\}) = \{p,q\}$	$E(\{q,r\}) = \{p,q,r\}$	$E(\{r\}) = \{p,q,r\}$
$*r$	$E(\{p,q,r\}) = \{p,q,r\}$	$E(\{q,r\}) = \{p,q,r\}$	$E(\{r,p\}) = \{p,q,r\}$

$F' = \{r\}$ (neither $E(p)$ nor $E(q)$ contains r).

Step 2: subset construction from $\{p\}$

$\delta_D(R,x) = \bigcup_{s\in R} \delta'(s,x)$, reachable subsets only:

	a	b	c
$\to A = \{p\}$	$\{p\} = A$	$\{p,q\} = B$	$\{p,q,r\} = C$
$B = \{p,q\}$	$\{p,q\} = B$	$\{p,q\} \cup \{p,q,r\} = C$	C
$*C = \{p,q,r\}$	C	C	C



So $L = \{w \in \{a,b,c\}^* \mid w$ contains a c or at least two b's$\}$.

Accepted strings of length ≤ 3 (general forms)

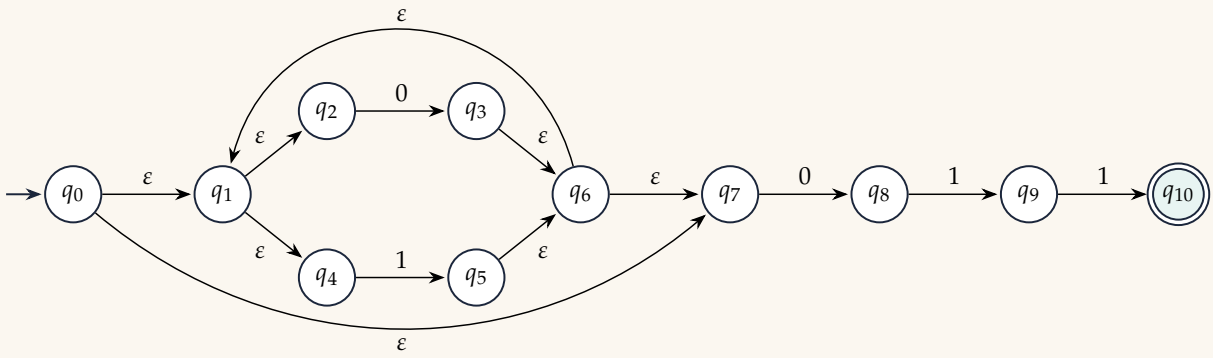
Reading a never changes the DFA state; one b moves $\{p\} \to \{p,q\}$; a second b or any c reaches the accepting sink.

- Length 0, 1: ε is rejected; of length 1 only c.
- Length 2: xc and cx for any $x \in \{a,b,c\}$, and bb — i.e. $\{ac, bc, cc, ca, cb, bb\}$.
- Length 3: *every* string containing at least one c (19 strings: Σ^3 minus the 8 strings over $\{a,b\}$), plus abb, bab, bba, bbb (the strings over $\{a,b\}$ with ≥ 2 b's) — 23 strings in total.

28. 2020 (Jan term) · Q4(b) · 20 marks · ★★★ High · Trace · L9

January 2020 Term • Q4(b) Source M37 **MARKS 20**

Convert the shown NFA to an equivalent DFA, showing computations.



SOLUTION

(This NFA recognizes $(0 \cup 1)^*011$. Numbers abbreviate q_i.)

Step 1: remove the ε-transitions

$\delta'(q,x) = E\big(\bigcup_{r\in E(q)} \delta(r,x)\big)$, $F' = \{q \mid E(q) \ni q_{10}\} = \{q_{10}\}$ (no other state reaches q_{10} by ε-moves alone):

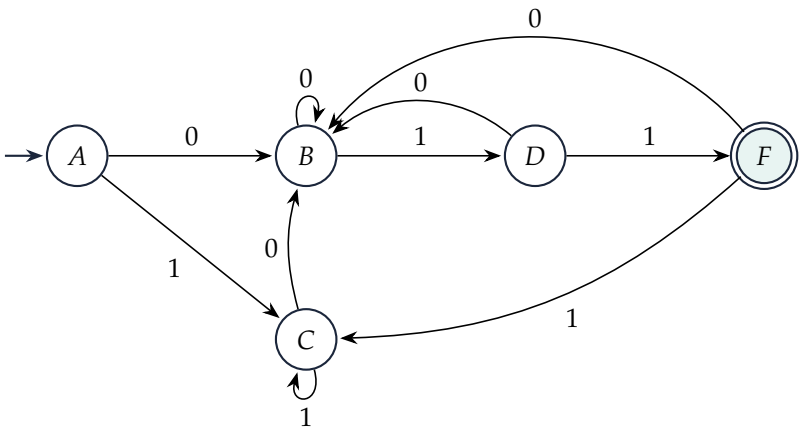
q	$E(q)$	$\delta'(q,0)$	$\delta'(q,1)$
$\to 0$	$\{0,1,2,4,7\}$	$E(\{3,8\}) = \{1,2,3,4,6,7,8\}$	$E(\{5\}) = \{1,2,4,5,6,7\}$
1	$\{1,2,4\}$	$E(\{3\}) = \{1,2,3,4,6,7\}$	$\{1,2,4,5,6,7\}$
2	$\{2\}$	$\{1,2,3,4,6,7\}$	$\emptyset$
3	$\{1,2,3,4,6,7\}$	$\{1,2,3,4,6,7,8\}$	$\{1,2,4,5,6,7\}$
4	$\{4\}$	$\emptyset$	$\{1,2,4,5,6,7\}$
5	$\{1,2,4,5,6,7\}$	$\{1,2,3,4,6,7,8\}$	$\{1,2,4,5,6,7\}$
6	$\{1,2,4,6,7\}$	$\{1,2,3,4,6,7,8\}$	$\{1,2,4,5,6,7\}$
7	$\{7\}$	$\{8\}$	$\emptyset$
8	$\{8\}$	$\emptyset$	$\{9\}$
9	$\{9\}$	$\emptyset$	$\{10\}$
$*10$	$\{10\}$	$\emptyset$	$\emptyset$

Step 2: subset construction from $\{q_0\}$

$\delta_D(R,x) = \bigcup_{r\in R} \delta'(r,x)$, reachable subsets only:

DFA state	on 0	on 1
$\to A = \{0\}$	$\{1,2,3,4,6,7,8\} = B$	$\{1,2,4,5,6,7\} = C$
$B = \{1,2,3,4,6,7,8\}$	B	$\{1,2,4,5,6,7\} \cup \{9\} = \{1,2,4,5,6,7,9\} = D$
$C = \{1,2,4,5,6,7\}$	B	C
$D = \{1,2,4,5,6,7,9\}$	B	$\{1,2,4,5,6,7\} \cup \{10\} = \{1,2,4,5,6,7,10\} = F$
$*F = \{1,2,4,5,6,7,10\}$	B	C

Only F contains $q_{10} \in F'$, so F is the sole accepting state.



29. 2019-20 · Q4(a) · 5 marks · ★★★ High · Theory · L9

2019–2020 • Q4(a) Source M41 **MARKS 5**

When a five-state NFA is converted to a DFA, what are the minimum and maximum possible numbers of DFA states? Justify.

SOLUTION

Each DFA state of the subset construction is a subset of the 5 NFA states, so there are at most $2^5 = \mathbf{32}$ states (counting $\emptyset$). The bound is attainable. Example: states $0, \ldots, 4$, start 0, alphabet $\{a, b, c\}$ with a: $i \mapsto \{i+1 \bmod 5\}$; b: $0 \mapsto \{0,1\}$, $i \mapsto \{i\}$ for $i \neq 0$; c: $0 \mapsto \emptyset$, $i \mapsto \{i\}$ for $i \neq 0$. In the subset construction a rotates a subset, b adds 1 to a subset containing 0, and c deletes 0. From $\{0\}$ the word $(ba^4)^3 b$ reaches $\{0,1,2,3,4\}$, and any element j can then be deleted by $a^{5-j}ca^j$ (rotate j to 0, delete, rotate back). So all 32 subsets are reachable DFA states. At least **1** state is always needed, and 1 can happen: e.g. an NFA whose start state is accepting with self-loops on every symbol (accepts Σ^*) yields the single DFA state $E(\{q_0\})$, which maps to itself.

Answer. Minimum 1, maximum $2^5 = 32$.

30. 2017-18 · Q4(a) · 15 marks · ★★ Medium · Trace · L9

Same question — solution already given under #27.

4 Regular Languages — Lec05 — Closure Properties and Regular Expressions

Notes: CSE211_Hakim_Regular_Languages_Notes.pdf

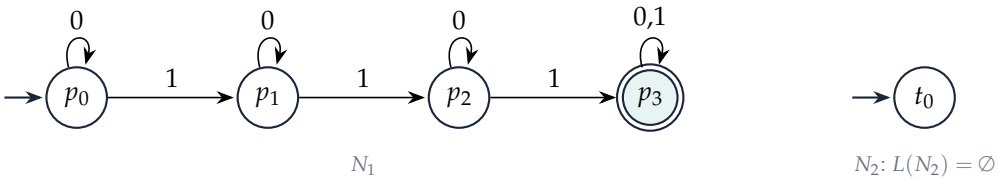
31. 2023-24 · Q1(c) · 10 marks · ★★★ High · Design · L10

2023–2024 • Q1(c) Source N01 — MARKS 10

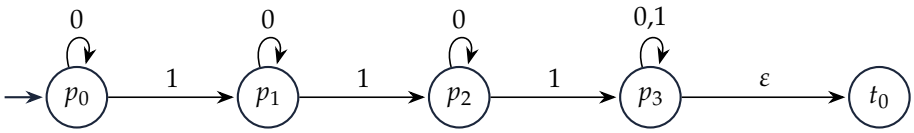
Draw NFAs for the concatenation of two languages over $\{0,1\}$: (i) strings containing at least three 1's and (ii) the empty set. Begin with the individual NFAs and apply the construction used to prove regular-language closure under concatenation.

SOLUTION

Individual NFAs. N_1 (at least three 1's) and N_2 (the empty set: one non-accepting start state, no transitions):



Concatenation construction. Start state = start of N_1; add an ε-edge from every accepting state of N_1 to the start of N_2; the accepting states of N are those of N_2 only (the accepting states of N_1 lose their status).



N has **no accepting state**, so $L(N) = \emptyset$ — as it must be, since $L \circ \emptyset = \emptyset$ for every L (there is no string of $\emptyset$ to append).

32. 2023-24 · Q3(b) · 13 marks · ★★★ High · Theory · L10

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q3(b) Source N02 — MARKS 13

In proving closure under star, someone proposes marking the old start state accepting, adding no new start state, and leaving the rest of the construction unchanged. Explain carefully why this is a bad idea and give a counterexample.

2016–2017 • Q4(a) Source M30 — MARKS 10

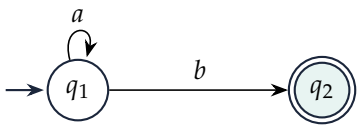
In the usual NFA construction for A^*, add a new start state that is also accepting, plus the required ε-edges. Hastif Hasty proposes instead marking the original start state accepting and keeping the other new edges where applicable. Explain clearly when this proposed construction fails.

SOLUTION

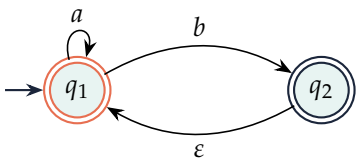
Correct construction. Given N_1 for A: add a new start state q_0 which is accepting, $q_0 \xrightarrow{\varepsilon} q_1$ (old start), and ε-edges from every accepting state back to q_1. The new state exists only to accept ε and has *no incoming edges*.

Proposal. Make the old start q_1 accepting (to get ε) and keep the back ε-edges $F_1 \to q_1$. The danger: q_1 *may have incoming edges inside* N_1. Then a partial computation that returns to q_1 without completing a word of A is now accepted.

Counterexample. $A = a^*b$, with N_1: $q_1 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_2$, q_2 accepting.



N_1: $L = a^*b$



proposed: accepts $a, aa, \ldots$

$A^* = (a^*b)^*$ consists of ε and strings ending in b. The proposed machine accepts a (loop on the now-accepting q_1), but $a \notin A^*$. So the construction is wrong.

When does it fail / work? It is guaranteed safe when the old start state has *no incoming transitions* (no path returns to it except via the new ε-edges). If q_1 has incoming transitions it can fail: any string that brings N_1 back to q_1 without finishing a word of A (like a above) is wrongly accepted. Adding a fresh start state avoids this in every case.

33. 2022-23 · Q3(c) · 8 marks · ★★★ High · Theory · L10

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2022–2023 • Q3(c) Source M51 — MARKS 8

Explain in detail why the Kleene star of a null set is an empty string.

2021–2022 • Q3(a) Source M45 — MARKS 4

Why is the Kleene star of the null set an empty string?

2016–2017 • Q2(c) Source M29 — MARKS 10

Why is the Kleene star of the null set an empty string? Explain clearly.

SOLUTION

Precisely: $\emptyset^* = \{\varepsilon\}$ — the language containing only the empty string (not $\emptyset$, and not the string ε itself).

From the definition. For any language L,

$$L^* = \bigcup_{k \geq 0} L^k = L^0 \cup L^1 \cup L^2 \cup \cdots, \qquad L^0 = \{\varepsilon\}, \quad L^{k+1} = L^k \circ L.$$

$L^0 = \{\varepsilon\}$ for *every* L: concatenating zero strings gives the empty string, and $\{\varepsilon\}$ is the identity of concatenation ($\{\varepsilon\} \circ L = L$), just as $x^0 = 1$ in arithmetic. For $L = \emptyset$ and $k \geq 1$, $\emptyset^k = \emptyset$ because a concatenation needs a string from each factor and $\emptyset$ has none ($\emptyset \circ M = \emptyset$, like multiplying by 0). Hence

$$\emptyset^* = \{\varepsilon\} \cup \emptyset \cup \emptyset \cup \cdots = \{\varepsilon\}.$$

Intuition. L^* = "choose any number of strings from L and glue them". From $\emptyset$ you can choose zero strings only, and gluing nothing yields ε.
Automaton view. The star construction adds a new start state that is accepting. Applied to an NFA for $\emptyset$ (no accepting states), the only accepted string is ε.

34. 2021-22 · Q2(c) · 6 marks · ★★★ High · Theory · L10

2021–2022 • Q2(c) Source M45 — MARKS 6

Over $\{a, b\}$, let $L_1 = \{a, aa, aaa\}$, $L_2 = \{a, ab, aab, aabb\}$, and $L_3 = \{w \mid$ the number of b's is divisible by $3\}$. State whether each is finite or infinite, with reasons: (i) $L_1^* \circ L_2^*$; (ii) $L_1 \circ (L_1 \cup L_2)$; (iii) $L_2 \circ L_3$.

SOLUTION

(i) Infinite. $L_1^* \supseteq \{a^n \mid n \geq 0\}$ and $\varepsilon \in L_2^*$, so $a^n \in L_1^* L_2^*$ for every n. (Star of a language containing a non-empty string is always infinite.)

(ii) Finite. Concatenation of finite languages is finite: $|L_1 \circ (L_1 \cup L_2)| \leq 3 \cdot 6 = 18$. (In fact it has 12 distinct strings: $a^2, \ldots, a^6$, $a^k b$ for $2 \leq k \leq 5$, and $a^k bb$ for $3 \leq k \leq 5$.)

(iii) Infinite. L_3 is infinite ($a^n \in L_3$ for all n, also $b^3, b^6, \ldots$) and $L_2 \neq \emptyset$; e.g. $a \cdot a^n \in L_2 L_3$ for all n.

35. 2021-22 · Q3(a) · 4 marks · ★★★ High · Theory · L10

Same question — solution already given under #33.

36. 2021-22 · Q4(b) · 12 marks · ★★★ High · Theory · L10

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q4(b) Source M46 MARKS 12

Given NFAs $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ for L_1 and $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ for L_2, construct $N = (Q, \Sigma, \delta, q_0, F)$ for $L_1 \cup L_2$. Explain the reasons for: (i) choosing its start state; (ii) choosing its accepting states; (iii) each case for transitions from the new start state and their possible destinations.

2019–2020 • Q3(c) Source M41 MARKS 15

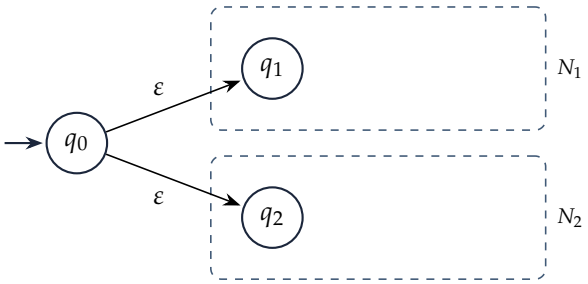
NFAs N_1 and N_2 recognize L_1 and L_2. To construct $N = (Q, \Sigma, \delta, q_0, F)$ recognizing $L_1 \cup L_2$, write the expression for δ and explain each part with justification.

SOLUTION

Assume $Q_1 \cap Q_2 = \emptyset$ (rename if necessary).

$$Q = \{q_0\} \cup Q_1 \cup Q_2, \qquad F = F_1 \cup F_2,$$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \\ \delta_2(q,a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0,\ a = \varepsilon \\ \emptyset & q = q_0,\ a \neq \varepsilon \end{cases} \qquad (a \in \Sigma_\varepsilon).$$



(i) Start state. A *new* state q_0 is needed because the machine must be able to begin either computation; we cannot simply pick q_1 or q_2. Being fresh, it has no incoming edges, so it cannot introduce unintended paths (and it does not disturb the internal behaviour of N_1 or N_2).

(ii) Accepting states $F = F_1 \cup F_2$. A string is in $L_1 \cup L_2$ iff it is accepted by N_1 or by N_2, i.e. iff some branch ends in F_1 or in F_2. q_0 is not accepting; if $\varepsilon \in L_1 \cup L_2$ it is still accepted via the ε-move into q_1 or q_2.

(iii) Transitions. For $q \in Q_1$ (resp. Q_2) we copy δ_1 (resp. δ_2), so each sub-machine runs unchanged. From q_0: on ε go to *both* q_1 and q_2 — nondeterministically guessing which language w belongs to, without consuming input; on any real symbol $a \neq \varepsilon$ there is no move ($\emptyset$), because consuming a symbol at q_0 would skip the first symbol of w and corrupt both simulations.

Correctness. $w \in L(N) \iff$ some run $q_0 \xrightarrow{\varepsilon} q_i \rightsquigarrow F_i \iff w \in L(N_1) \cup L(N_2)$.

37. 2020 (Jan term) · Q3(b) · 20 marks · ★★★ High · Theory · L10

January 2020 Term • Q3(b) Source M37 — MARKS 20

NFA $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognizes L_1. Construct $N = (Q, \Sigma, \delta, q_0, F)$ recognizing L_1^*. For $q \in Q$ and $a \in \Sigma_\varepsilon$, δ is defined by: $\delta_1(q,a)$ if $q \in Q_1, q \notin F_1$; $\delta_1(q,a)$ if $q \in F_1, a \neq \varepsilon$; $\delta_1(q,a) \cup \{q_1\}$ if $q \in F_1, a = \varepsilon$; $\{q_1\}$ if $q = q_0, a = \varepsilon$; $\emptyset$ if $q = q_0, a \neq \varepsilon$. Explain in plain English the purpose of each condition.

SOLUTION

Here $Q = \{q_0\} \cup Q_1$, the new start q_0, and $F = \{q_0\} \cup F_1$.

(i) $q \in Q_1, q \notin F_1$: $\delta_1(q,a)$. Inside a piece that is not yet finished, behave exactly like N_1 — we are in the middle of reading one word of L_1.

(ii) $q \in F_1, a \neq \varepsilon$: $\delta_1(q,a)$. At an accepting state, N_1 may still continue reading (the current word of L_1 might be a prefix of a longer word of L_1), so its ordinary moves are kept.

(iii) $q \in F_1, a = \varepsilon$: $\delta_1(q,\varepsilon) \cup \{q_1\}$. Having completed one word of L_1, the machine may (without reading input) jump back to N_1's start to begin the *next* word — this is the "repeat" of the star. The original ε-moves of q are kept too.

(iv) $q = q_0, a = \varepsilon$: $\{q_1\}$. The new start state immediately enters N_1 to read the first piece. Because q_0 is accepting, zero pieces (ε) are accepted — required since $\varepsilon \in L_1^*$.

(v) $q = q_0, a \neq \varepsilon$: $\emptyset$. q_0 consumes no input; it exists only to accept ε and to hand over to q_1. Having no other edges (and no incoming edges) guarantees it adds no unintended strings — which is why we do not simply make q_1 accepting (see the counterexample in 2023–24 Q3(b) above).

38. 2019-20 · Q3(c) · 15 marks · ★★★ High · Theory · L10

Same question — solution already given under #36.

39. 2022-23 · Q3(b) · 12 marks · ★★★ High · Theory · L11

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2022–2023 • Q3(b) Source M51 — MARKS 12

Swapping accepting and nonaccepting states of a DFA for L yields a DFA for its complement. Does this always work for an NFA? Give an example.

2021–2022 • Q1(c) Source M44 — MARKS 6 + 3 = 9

Explain why swapping accepting and nonaccepting states of a DFA recognizing C gives a DFA for the complement of C. Does the same method work for NFAs? Explain with examples.

2017–2018 • Q2(a) Source M32 — MARKS 10

Given a DFA D for L over Σ, how can you obtain a DFA for $\Sigma^* - L$? Explain why the method works and when it would fail. Does it work for NFAs? Explain.

2015–2016 • Q3(b) Source M25 — MARKS 5 1/3

Swapping accepting and nonaccepting states in a DFA gives its complement. Give an NFA example for which the same swap does not recognize the complement.

SOLUTION

Method. From $D = (Q, \Sigma, \delta, q_0, F)$ build $\overline{D} = (Q, \Sigma, \delta, q_0, Q - F)$.

Why it works for DFAs. δ is a total function, so on every input w there is *exactly one* run and it ends in exactly one state $\hat{\delta}(q_0, w)$. Then

$$w \in L(\overline{D}) \iff \hat{\delta}(q_0, w) \in Q - F \iff \hat{\delta}(q_0, w) \notin F \iff w \notin L(D).$$

When it fails (even for "DFAs"). (1) If the DFA is *incomplete* (some transitions missing, implicitly rejecting), strings that "fall off" are rejected both before and after the swap. Fix: first add an explicit dead state receiving all missing moves (it becomes accepting after the swap). (2) If D is over a smaller alphabet than Σ, add the missing symbols (to the dead state) first.
NFAs: it does not work in general. An NFA accepts if *some* run ends in F. After swapping, it accepts if *some* run ends outside F — that is not the same as "*no* run ends in F". A string can have both kinds of runs and then is accepted by both machines.
Example. Over $\{a\}$: $q_0 \xrightarrow{a} q_0$, $q_0 \xrightarrow{a} q_1$, $F = \{q_1\}$; $L(N) = a^+$.

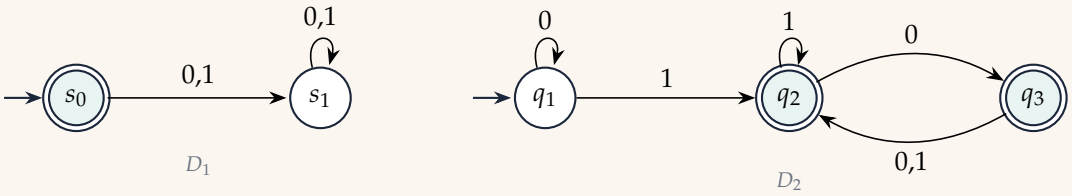


The complement of a^+ is $\{\varepsilon\}$, but the swapped NFA accepts a^* (e.g. a via the run staying in q_0). Missing transitions cause the same problem: dead branches are rejected in both versions.
Correct way for an NFA: convert it to a complete DFA by the subset construction, *then* swap.

40. 2022-23 · Q4(a) · 10 marks · ★★★ High · Design · L11

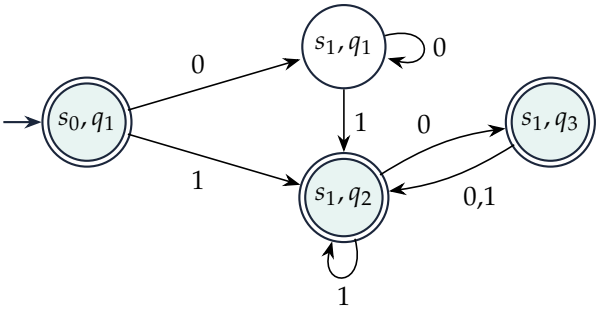
2022–2023 • Q4(a) Source M51 MARKS 10

Using basic principles, construct a DFA D for $L(D_1) \cup L(D_2)$, where D_1 and D_2 are shown. (i) Describe $L(D_1)$ in plain English. (ii) Draw the start state of D and two states reachable from it.



SOLUTION

(i) D_1 accepts only in s_0 and leaves it on any symbol forever, so $L(D_1) = \{\varepsilon\}$ — only the empty string. (For reference, $L(D_2)$ = strings containing at least one 1.)
(ii) Product construction. $Q = Q_1 \times Q_2$, start (s_0, q_1), $\delta((r,s),x) = (\delta_1(r,x), \delta_2(s,x))$, and $F = \{(r,s) \mid r \in F_1 \text{ or } s \in F_2\}$ for union.



The start state (s_0, q_1) is accepting (because $s_0 \in F_1$); the two states reachable in one step are (s_1, q_1) (non-accepting) and (s_1, q_2) (accepting). The complete D has the 4 reachable states shown and recognises $\{\varepsilon\} \cup \{w \mid w \text{ contains a } 1\}$.

41. 2021-22 · Q1(c) · 9 marks · ★★★ High · Theory · L11

Same question — solution already given under #39.

42. 2017-18 · Q2(a) · 10 marks · ★★ Medium · Theory · L11

Same question — solution already given under #39.

43. 2023-24 · Q4(b) · 12 marks · ★★★ High · Theory · L12

2023–2024 • Q4(b) Source N02 — MARKS 12

For strings over $\{a, b\}$ containing no more than one occurrence of aa, determine which options are correct. Give reasons for every option: A. $(b + ab)^*(\varepsilon + a + aa)(b + ab)^*$; B. $(b + ab)^*(\varepsilon + a + aa)(b + ba)^*$; C. $(b + ab)^*(\varepsilon + a)(b + ba)^*$; D. $(b + ba)^*(\varepsilon + a + aa)(b + ba)^*$.

SOLUTION

Useful facts: $(b + ab)^*$ = strings with no aa that are empty or *end with* b; $(b + ba)^*$ = strings with no aa that are empty or *start with* b. Occurrences are counted with overlap (aaa contains aa twice).

× **A.** The right factor may begin with a: $aa \cdot ab = aaab$ contains aa twice (positions 1–2 and 2–3). Over-generates.

✓ **B.** *Sound*: the left part ends in b (or is empty) and the right part starts with b (or is empty), so the middle ε, a or aa is isolated; at most one aa arises. *Complete*: if w has exactly one aa, write $w = u\,aa\,v$; u cannot end in a (else aaa) so $u \in (b + ab)^*$, and v cannot start with a so $v \in (b + ba)^*$. If w has no aa: if it ends in a, $w = u\,a$ with $u \in (b + ab)^*$; otherwise $w \in (b + ab)^*$.

× **C.** Sound but incomplete: it generates only strings with *zero* occurrences, missing e.g. aa, $baab$.

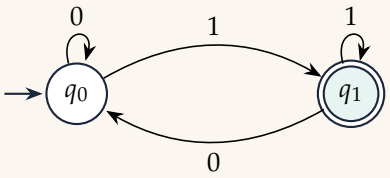
× **D.** The left factor may end in a: $ba \cdot aa = baaa$ contains aa twice. Over-generates.

Answer. Only B is correct. (Verified by enumerating all strings of length ≤ 9.)

44. 2022-23 · Q4(c) · 10 marks · ★★★ High · Theory · L12

2022–2023 • Q4(c) Source M51 MARKS 10

Which option(s) describe the shown DFA? Explain each choice as correct or incorrect: A. $0^*1(1+00^*1)^*$; B. $0^*1^*1+11^*0^*1$; C. $(0+1)^*1$.



SOLUTION

Every 1 leads to q_1 and every 0 to q_0, so the DFA accepts exactly the strings **ending in 1**.

✓ **A.** 0^*1 reaches q_1; afterwards each block is either 1 (loop) or 0^+1 (go to q_0, loop, come back). Any string ending in 1 splits uniquely this way. Correct.

× **B.** 0^*1^*1 has no 0 after a 1; 11^*0^*1 has at most one 0-block. It misses e.g. 0101 and 10101. Incorrect (it is a proper subset).

✓ **C.** Literally "anything, then 1". Correct.

Answer. A and C.

45. 2021-22 · Q3(d) · 12 marks · ★★★ High · Theory · L12

2021–2022 • Q3(d) Source M46 MARKS 12

Which regular expression(s) represent binary strings having both consecutive 00 and consecutive 11? Explain why each option is correct or incorrect: A. $(0+1)^*0011(0+1)^*+(0+1)^*1100(0+1)^*$; B. $(0+1)^*(00(0+1)^*11+11(0+1)^*00)(0+1)^*$; C. $(0+1)^*00(0+1)^*+(0+1)^*11(0+1)^*$; D. $00(0+1)^*11+11(0+1)^*00$.

SOLUTION

× **A.** Requires 00 and 11 to be *adjacent*. Misses 001011 (has 00 and 11 but neither 0011 nor 1100).

✓ **B.** Any string with both has an occurrence of 00 and one of 11; they cannot overlap, so one comes first: $x\,00\,y\,11\,z$ or $x\,11\,y\,00\,z$. Exactly what B says.

× **C.** "00 *or* 11": generates 00, which has no 11.

× **D.** Forces the string to start with one pair and end with the other; misses 00110, 10011.

Answer. Only B.

46. 2020 (Jan term) · Q2(c) · 10 marks · ★★★ High · Design · L12

January 2020 Term • Q2(c) Source M36 MARKS 10

Write a regular expression for the strings NOT belonging to the language of the DFA in 2(a) (the DFA accepting strings ending in 00 or 11).

SOLUTION

$L = (0 \cup 1)^*(00 \cup 11)$. Its complement: strings of length < 2, or whose last two symbols differ.

Answer. $\varepsilon \cup 0 \cup 1 \cup (0 \cup 1)^*(01 \cup 10)$

(Equivalently: make q_0, q_1, q_2 accepting and q_3, q_4 non-accepting in the given DFA and convert by GNFA state ripping.)

47. 2020 (Jan term) · Q3(a) · 10 marks · ★★★ High · Trace · L12

January 2020 Term • Q3(a) Source M37 MARKS 10

Evaluate each regular expression, showing detailed computations: (i) $\emptyset^*\emptyset^*\emptyset^*\emptyset^*$; (ii) $\emptyset^* \cup \{\varepsilon\}$; (iii) $(\emptyset^* \cup 1)\{0,1\}$; (iv) $(11)^*0(00)^*$; (v) $(\emptyset^* \cup 0)\{0,1,11\}$.

SOLUTION

Recall $\emptyset^* = \{\varepsilon\}$ and $\{\varepsilon\}L = L$.

(i) $\{\varepsilon\}\{\varepsilon\}\{\varepsilon\}\{\varepsilon\} = \{\varepsilon\}$.

(ii) $\{\varepsilon\} \cup \{\varepsilon\} = \{\varepsilon\}$.

(iii) $(\{\varepsilon\} \cup \{1\})\{0,1\} = \{\varepsilon, 1\}\{0,1\} = \{0, 1, 10, 11\}$.

(iv) $\{1^{2i}\,0\,0^{2j} \mid i, j \geq 0\} = \{1^{2i}0^{2j+1}\}$: an even number of 1's followed by an odd number of 0's, e.g. 0, 110, 000, 11000.

(v) $\{\varepsilon, 0\}\{0, 1, 11\} = \{0, 1, 11, 00, 01, 011\}$.

48. 2020 (Jan term) · Q4(a) · 10 marks · ★★★ High · Theory · L12

January 2020 Term • Q4(a) Source M37 MARKS 10

Describe in English the language of $(1 + \varepsilon)(00^*1)^*0^*$.

SOLUTION

An optional leading 1, then blocks each consisting of one or more 0's followed by a single 1, then any trailing 0's. Thus every 1 except possibly the first symbol is immediately preceded by a 0, i.e. no two 1's are adjacent; conversely every string without 11 splits this way.

Answer. All binary strings with no two consecutive 1's.

49. 2019-20 · Q2(b) · 15 marks · ★★★ High · Trace · L12

2019–2020 • Q2(b) Source M40 MARKS 15

Simplify each regular expression to an equivalent expression, with explanations: (i) $\emptyset^* \cup a^* \cup b^* \cup (a \cup b)^*$; (ii) $((a^*b^*)^*(b^*a^*)^*)^*$; (iii) $(a^*b)^* \cup (b^*a)^*$.

SOLUTION

(i) $\emptyset^* = \{\varepsilon\}$, and $\{\varepsilon\}$, a^*, b^* are all subsets of $(a \cup b)^*$. A union with a superset is the superset:

Answer. $(a \cup b)^*$.

(ii) a^*b^* contains a and b, so $(a^*b^*)^* \supseteq (a \cup b)^*$, hence $= (a \cup b)^*$. Likewise $(b^*a^*)^* = (a \cup b)^*$. Then $(a \cup b)^*(a \cup b)^* = (a \cup b)^*$ and its star is itself:

Answer. $(a \cup b)^*$.

(iii) $(a^*b)^* = \varepsilon$ plus every string ending in b (split after each b). $(b^*a)^* = \varepsilon$ plus every string ending in a. Every string is empty or ends in a or b:

Answer. $(a \cup b)^*$.

50. 2019-20 · Q4(b) · 15 marks · ★★★ High · Design · L12

2019–2020 • Q4(b) Source M41 MARKS 15

Over $\{a, b\}$, write regular expressions for (i) the number of a's divisible by 3; (ii) no more than three a's; (iii) exactly one occurrence of substring aaa.

SOLUTION

(i) Group the a's in threes, with b's anywhere:

Answer. $b^*(ab^*ab^*ab^*)^*$

(ii) At most three optional a's separated by b's:

Answer. $b^*(a \cup \varepsilon)b^*(a \cup \varepsilon)b^*(a \cup \varepsilon)b^*$

(iii) "Exactly one occurrence" (overlapping counted, so $aaaa$ has two) means exactly one maximal a-block of length exactly 3 and all other a-blocks of length ≤ 2. Before the block: no aaa and ending in b or empty $= (b \cup ab \cup aab)^*$; after: no aaa and starting with b or empty $= (b \cup ba \cup baa)^*$.

Answer. $(b \cup ab \cup aab)^*\, aaa\, (b \cup ba \cup baa)^*$

(All three verified by enumeration up to length 10.)

51. 2017-18 · Q4(b) · 20 marks · ★★ Medium · Design · L12

2017–2018 • Q4(b) Source M33 — MARKS 20

Find regular expressions for: (i) $\{w \in \{a,b,c\}^* \mid \text{each alphabet symbol occurs at least once}\}$; (ii) $\{a^n b^m \mid n \geq 3,\ m \text{ odd}\}$; (iii) $\{a^n b^m \mid n+m \text{ odd}\}$; (iv) $\{vwv \mid v,w \in \{a,b\}^*,\ |v|=2\}$; (v) $\{w \in \{a,b\}^* \mid n_a(w) \bmod 3 = 0\}$.

SOLUTION

Let $\Sigma = (a \cup b \cup c)$.

(i) Split on the order of first occurrences (6 permutations):

$$\Sigma^* a \Sigma^* b \Sigma^* c \Sigma^* \cup \Sigma^* a \Sigma^* c \Sigma^* b \Sigma^* \cup \Sigma^* b \Sigma^* a \Sigma^* c \Sigma^*$$
$$\cup\, \Sigma^* b \Sigma^* c \Sigma^* a \Sigma^* \cup \Sigma^* c \Sigma^* a \Sigma^* b \Sigma^* \cup \Sigma^* c \Sigma^* b \Sigma^* a \Sigma^*$$

(ii) $aaaa^*\, b(bb)^*$

(iii) n odd & m even, or n even & m odd: $a(aa)^*(bb)^* \cup (aa)^* b(bb)^*$

(iv) $v \in \{aa, ab, ba, bb\}$: $aa(a \cup b)^* aa \cup ab(a \cup b)^* ab \cup ba(a \cup b)^* ba \cup bb(a \cup b)^* bb$

(v) $b^*(ab^*ab^*ab^*)^*$

52. 2021-22 · Q2(b) · 10 marks · ★★★ High · Design · L13

2021–2022 • Q2(b) Source M45 — MARKS 10

Using basic principles, convert $(((00)^*(11)) \cup 01)^*$ into an NFA.

SOLUTION

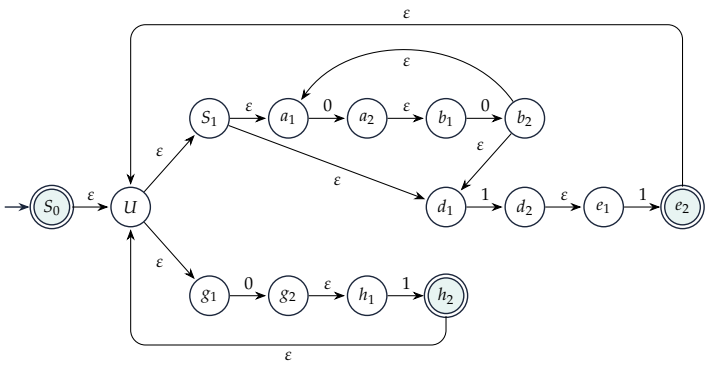
Build bottom-up with the slide (Sipser) constructions: a symbol x is the two-state NFA $q \xrightarrow{x} q'$ (q' accepting), and every concatenation joins the pieces by an ε-edge.

(i) 00: $a_1 \xrightarrow{0} a_2 \xrightarrow{\varepsilon} b_1 \xrightarrow{0} b_2$ (accepting b_2). $(00)^*$: new accepting start S_1, $S_1 \xrightarrow{\varepsilon} a_1$, $b_2 \xrightarrow{\varepsilon} a_1$; accepting S_1, b_2.

(ii) 11: $d_1 \xrightarrow{1} d_2 \xrightarrow{\varepsilon} e_1 \xrightarrow{1} e_2$. $(00)^*11$: ε-edges from the accepting states S_1, b_2 of $(00)^*$ to d_1; only e_2 accepts.

(iii) 01: $g_1 \xrightarrow{0} g_2 \xrightarrow{\varepsilon} h_1 \xrightarrow{1} h_2$. Union: new start U with $U \xrightarrow{\varepsilon} S_1$, $U \xrightarrow{\varepsilon} g_1$; accepting e_2, h_2.

(iv) Star: new accepting start S_0, $S_0 \xrightarrow{\varepsilon} U$, and $e_2 \xrightarrow{\varepsilon} U$, $h_2 \xrightarrow{\varepsilon} U$. Accepting: S_0, e_2, h_2.



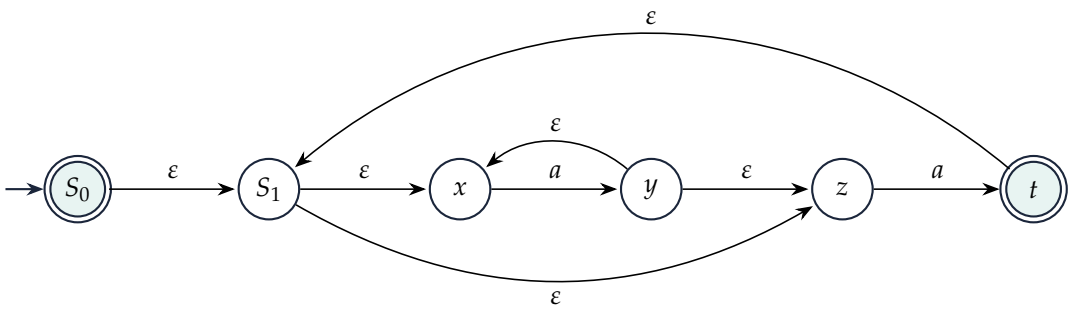
53. 2019-20 · Q3(a) · 5 marks · ★★★ High · Design · L13

2019–2020 • Q3(a) Source M40 — MARKS 5

Construct an NFA for $((a^*)a)^*$ using basic principles.

SOLUTION

a: $x \xrightarrow{a} y$. a^*: new accepting start S_1, $S_1 \xrightarrow{\varepsilon} x$, $y \xrightarrow{\varepsilon} x$. $(a^*)a$: ε from S_1, y to $z \xrightarrow{a} t$. $((a^*)a)^*$: new accepting start $S_0 \xrightarrow{\varepsilon} S_1$, $t \xrightarrow{\varepsilon} S_1$.



(The language is simply a^*, but the question asks for the systematic construction.)

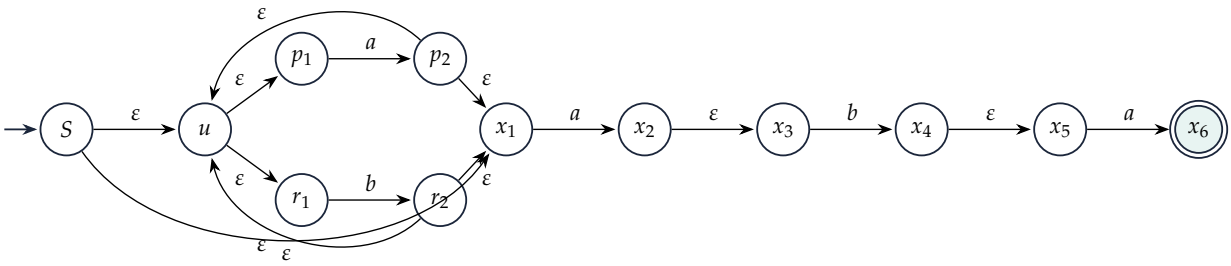
54. 2017-18 · Q2(c) · 8 marks · ★★ Medium · Design · L13

2017–2018 • Q2(c) Source M32 — MARKS 8

Using regular-language closure constructions, draw an NFA for $(a \cup b)^*aba$.

SOLUTION

$a \cup b$: start u with $u \xrightarrow{\varepsilon} p_1 \xrightarrow{a} p_2$ and $u \xrightarrow{\varepsilon} r_1 \xrightarrow{b} r_2$. Star: new accepting start S, $S \xrightarrow{\varepsilon} u$, $p_2, r_2 \xrightarrow{\varepsilon} u$. Concatenate with a, b, a (each a two-state NFA) using ε-edges from the previous accepting states.



(After removing ε-moves this collapses to the familiar 4-state NFA $s \xrightarrow{a,b} s$, $s \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{a} 3$.)

5 Pumping Lemma — Lec06 — Non-Regular Languages

Notes: CSE211_Hakim_NonRegular_Notes.pdf

55. 2023-24 · Q2(a) · 15 marks · ★★★ High · Theory · L15

2023–2024 • Q2(a) Source N01 — MARKS 15

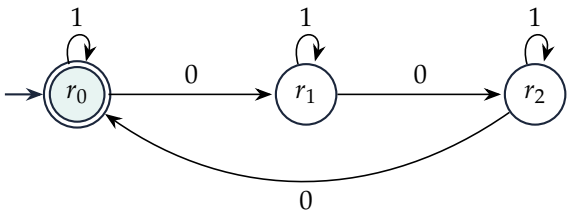
Justify why the regular pumping lemma requires $|xy| \leq p$ in $w = xyz$. Explain clearly with an example language and its finite automaton.

SOLUTION

Where the condition comes from. Let p be the number of states of a DFA for L and $s \in L$ with $|s| \geq p$. The run on s visits $p+1$ states $r_0, r_1, \ldots, r_p$ while reading just the *first* p *symbols*. By the pigeonhole principle two of these coincide, $r_j = r_k$ with $0 \leq j < k \leq p$. Take $x = s_1 \cdots s_j$, $y = s_{j+1} \cdots s_k$ (the loop), z = the rest. Because the repetition is found inside the first p symbols, $|xy| = k \leq p$ *automatically* — it costs nothing to state it, and it is true.

Why we need it. It pins the pumpable part y near the *start* of s, so in a proof we know what y consists of. Example $L = \{0^n 1^n\}$: assume L is regular and let p be its pumping length; choose $s = 0^p 1^p \in L$, $|s| = 2p \geq p$. Because $|xy| \leq p$, every split has $y = 0^k$ with $k \geq 1$, and $xy^2z = 0^{p+k}1^p \notin L$ — a contradiction, so L is not regular. Without $|xy| \leq p$ the adversary could choose $y = 0^m 1^m$ or y inside the 1's, and we would have to analyse many cases; and for some languages, where a later portion of s genuinely can be pumped, no contradiction could be reached at all.

Example automaton. $L = \{w \in \{0,1\}^* \mid \#_0(w) \equiv 0 \pmod 3\}$, DFA with $p = 3$ states:



For $s = 10010 \in L$ (three 0's, $|s| = 5 \geq 3$): the first $p = 3$ symbols give the run $r_0 \xrightarrow{1} r_0 \xrightarrow{0} r_1 \xrightarrow{0} r_2$; already r_0 repeats after the first symbol, so $x = \varepsilon$, $y = 1$, $z = 0010$, and $|xy| = 1 \leq 3$. Pumping y goes around the loop and stays in L. A loop is *always* found within the first p symbols, which is precisely $|xy| \leq p$.

56. 2021-22 · Q4(a) · 4 marks · ★★★ High · Theory · L15

2021–2022 • Q4(a) Source M46 — MARKS 4

Why is the regular pumping lemma useful for 'negative proofs' only?

SOLUTION

The lemma is an implication: *regular* $\Rightarrow$ *pumpable*. Its contrapositive, *not pumpable* $\Rightarrow$ *not regular*, is what we use. The converse is false: some nonregular languages satisfy the pumping property, e.g.

$$L = \{a^i b^j c^k \mid i, j, k \geq 0,\ i = 1 \Rightarrow j = k\}$$

is nonregular, yet every long string can be pumped with $p = 2$ (pump the first symbol unless $i = 2$, where pump the first two). So showing that a language *can* be pumped proves nothing; regularity must be shown positively by giving a DFA/NFA/regular expression or using closure properties.

57. 2023-24 · Q4(c) · 13 marks · ★★★ High · Proof · L16

2023–2024 • Q4(c) Source N02 — MARKS 13

Use the regular pumping lemma to show that $L = \{a^{n!} \mid n \geq 1\}$ is not regular.

SOLUTION

Assume L is regular and let p be its pumping length. Let $m = \max(p, 2)$ and $s = a^{m!} \in L$; $|s| = m! \geq m \geq p$. For any split, $y = a^k$ with $1 \leq k \leq |xy| \leq p \leq m$. Pump up ($i = 2$): $|xy^2z| = m! + k$. Now

$$m! < m! + k \leq m! + m < m! + m \cdot m! = (m+1)!,$$

(using $m \geq 2$, so $m < m \cdot m!$). Thus $m! + k$ lies strictly between two consecutive factorials and is not a factorial, so $xy^2z \notin L$. This contradicts the PL; L is not regular. ■

58. 2019-20 · Q4(c) · 15 marks · ★★★ High · Proof · L16

2019–2020 • Q4(c) Source M41 — MARKS 15

Use the pumping lemma to prove $L = \{a^{2n}b^{3n}a^n \mid n \geq 0\}$ is not regular.

SOLUTION

Assume L is regular and let p be its pumping length. Take $s = a^{2p}b^{3p}a^p \in L$ ($n = p$), $|s| = 6p \geq p$. Let $s = xyz$ be any split with $|y| > 0$, $|xy| \leq p$. Since $|xy| \leq p \leq 2p$, x and y lie inside the first block: $y = a^k$, $k \geq 1$. Pump up:

$$xy^2z = a^{2p+k}b^{3p}a^p.$$

For this to be in L, the last block forces $n = p$, so the first block would have to be a^{2p} — but it is a^{2p+k}. Hence $xy^2z \notin L$, contradicting the lemma. L is not regular. ■

59. 2022-23 · Q2(b) · 15 marks · ★★★ High · Proof · L17

2022–2023 • Q2(b) Source M50 — MARKS 15

For $L = \{w \mid w = a^{2+3k}$ or $w = b^{10+12k},\ k \geq 0\}$ over $\{a, b\}$, argue carefully using the pumping lemma whether 3, 5, 9, or 24 is a pumping length. Find and analyze the minimum pumping length.

SOLUTION

$L = \{a^2, a^5, a^8, \ldots\} \cup \{b^{10}, b^{22}, b^{34}, \ldots\}$. A number p is a pumping length if *every* $s \in L$ with $|s| \geq p$ has a split satisfying (1)–(3).

***a*-strings.** For $s = a^{2+3k}$, $y = a^j$. Pumping up keeps the length $\equiv 2 \pmod 3$ only if $3 \mid j$; pumping down needs $2 + 3k - j \geq 2$. So we need $y = a^3$ (or $a^6, \ldots$) and $k \geq 1$. With $x = \varepsilon$, $y = aaa$: $|xy| = 3$. This works for every a^{2+3k}, $k \geq 1$, whenever $p \geq 3$; and a^2 ($k = 0$) is unpumpable but is exempt as soon as $p \geq 3$.

***b*-strings.** For $s = b^{10+12k}$, $y = b^j$ must have $12 \mid j$, and pumping down needs $k \geq 1$. Hence (a) b^{10} can *never* be pumped, so we need $p > 10$; (b) y has length ≥ 12, so $|xy| \leq p$ forces $p \geq 12$. Conversely, if $p \geq 12$, every b-string with $|s| \geq p$ has $k \geq 1$ and the split $x = \varepsilon$, $y = b^{12}$ works: $xy^iz = b^{10+12(k-1+i)} \in L$ for all $i \geq 0$.

p	verdict
3	× $s = b^{10}$ (length $10 \geq 3$) cannot be pumped: $b^{10+j} \notin L$ for $1 \leq j \leq 3$
5	× same witness b^{10}
9	× same witness b^{10}
24	✓ a-strings: $y = a^3$; b-strings of length ≥ 24 are $b^{34}, b^{46}, \ldots$: $y = b^{12}$, $\lvert xy\rvert = 12 \leq 24$

Minimum. $p = 11$ fails: $s = b^{22}$ has length ≥ 11 but its y would need length 12 (or 24, …) > 11. $p = 12$ works by the analysis above.

Answer. 3, 5, 9 are not pumping lengths; 24 is. The minimum pumping length is 12.

60. 2021-22 · Q1(d) · 9 marks · ★★★ High · Proof · L17

2021–2022 • Q1(d) Source M44–M45 MARKS 9

Comment on the acceptability of these arguments by B. L. Perfunctory (BLP): (i) Assume binary palindromes are regular with pumping length p. Let $w = 00011000$. Since w is a palindrome, there are x, y, z satisfying the lemma. xy lies entirely in the initial 0^p, so x and y contain only zeros. Pumping down gives xz, which cannot be a palindrome, so the language is not regular. (ii) Assume equal-count binary strings are regular. Choose $w = 0^{p/4}1^{p/4}0^{p/4}1^{p/4} = xyz$. y lies entirely in the first zeros. For any permissible length of y, xy^2z has unequal counts and is outside the language; hence the language is not regular. (iii) Assume $\{0^{2n} \mid n \geq 0\}$ is regular. Choose $w = 0^{2p}$, $x = \varepsilon$, $y = 0$, $z = 0^{2p-1}$. Taking $i = 0$ gives $xy^iz = 0^{2p-1}$, whose length is odd; hence the language is not regular.

SOLUTION

(i) Not acceptable. BLP's string 00011000 is a *fixed* string; it must depend on p. If $p > 8$ the lemma says nothing about it (it is shorter than p), and "the initial 0^p" does not exist (it starts with only three 0's). The conclusion is true, but the proof is wrong. Fix: let p be the pumping length and $s = 0^p10^p$ (a palindrome, $|s| = 2p + 1 \geq p$); every split has $y = 0^k$, $k \geq 1$, and $xy^0z = 0^{p-k}10^p$ is not a palindrome — contradiction.

(ii) Not acceptable. BLP's string has length exactly p, so $|xy| \leq p$ allows y *anywhere* in it, not just in the first block. The adversary may pick $y = 01$ (straddling the first two blocks); then xy^iz keeps equal counts for all i and no contradiction arises. Fix: $s = 0^p1^p$ ($|s| = 2p \geq p$), forcing $y = 0^k$; then $xy^2z = 0^{p+k}1^p$ has unequal counts.

(iii) Not acceptable. The prover may not choose x, y, z; the lemma only promises that *some* split works. Here the adversary chooses $y = 00$ (legal whenever $p \geq 2$), and then $xy^iz = 0^{2p-2+2i} \in L$ for all i. In fact the language $(00)^*$ *is regular*, so no valid proof exists.

61. 2020 (Jan term) · Q1(b) · 12 marks · ★★★ High · Proof · L17

January 2020 Term • Q1(b) Source M36 MARKS 12

Surcharged Skye gives these pumping-lemma arguments. Explain clearly what, if anything, is wrong: (i) $L_1 = \{ww \mid w \in \{0,1\}^*\}$. Choose 0^p0^p, $x = \varepsilon$, $y = (00)^k$, $k \leq p/2$. Since $xy^iz \in L_1$ for all $i \geq 0$, L_1 is regular. (ii) $L_2 = \{w \in \{0,1\}^* \mid n_0(w) = 3n_1(w)\}$. Choose $0^{3p}1^p$ and $y = 0001$. Since $xy^iz \in L_2$ for all $i \geq 0$, we have failed to show that L_2 contradicts the lemma.

SOLUTION

(i) Wrong logic. The PL gives a *necessary* condition for regularity; showing that one string (or even every string) can be pumped never proves a language regular. Also, the prover does not get to choose y. In fact L_1 is *not* regular: let p be the pumping length and take $s = 0^p1\,0^p1 \in L_1$, $|s| = 2p + 2 \geq p$; every split has $y = 0^k$ ($k \geq 1$) in the first block, and $xy^2z = 0^{p+k}10^p1 \notin L_1$ (a square uu ending in 1 with two 1's needs u to contain exactly one 1, so both halves would be $0^{p+k}1 = 0^p1$, impossible). Contradiction.

(ii) Illegal split. With $s = 0^{3p}1^p$ and $|xy| \leq p$, y must lie within the first p symbols, which are all 0's — $y = 0001$ is not a legal choice (the first 1 is at position $3p + 1$). Moreover the prover must handle *every* legal y, not pick a convenient one. Correct argument: assume L_2 regular with pumping length p; $s = 0^{3p}1^p \in L_2$, $|s| = 4p \geq p$; every legal split has $y = 0^k$, $k \geq 1$; then $xy^2z = 0^{3p+k}1^p$ has $n_0 = 3p + k \neq 3p = 3n_1$, so $xy^2z \notin L_2$ — contradiction, and L_2 is not regular.

62. 2019-20 · Q1(a) · 5 marks · ★★★ High · Proof · L17

2019–2020 • Q1(a) Source M40 MARKS 5

For $L = \{a^n \mid n \text{ is prime}\}$, a pumping-lemma proof starts: 'Given p, let $w = a^p \ldots$', where p is the pumping length. Comment on the acceptability of this start.

SOLUTION

Not acceptable as written: the pumping length p is *any* number supplied by the lemma, not necessarily prime (e.g. $p = 4$), and then $a^p \notin L$, so the lemma does not apply to it. The chosen string must belong to L. Correct start: assume L is regular and let p be its pumping length; choose a prime $q > p$ (infinitely many primes exist) and $s = a^q \in L$, $|s| = q \geq p$. Every split has $y = a^k$, $1 \leq k \leq p$; pump $i = q + 1$: $|xy^{q+1}z| = q + qk = q(1 + k)$, a product of two numbers ≥ 2, hence not prime, so $xy^{q+1}z \notin L$ — contradiction, L is not regular. (If p happens to be prime, $s = a^p$ is fine, but the proof cannot assume that.)

63. 2017-18 · Q3(b) · 17 marks · ★★ Medium · Proof · L17

2017–2018 • Q3(b) Source M32–M33 MARKS 17

Surcharged Skye gives these pumping-lemma arguments. Explain clearly what, if anything, is wrong in each: (i) $L_1 = \{ww \mid w \in \{0,1\}^*\}$. Choose 0^p0^p, $x = \varepsilon$, $y = (00)^k$ with $k \leq p/2$. Since $xy^iz \in L_1$ for every $i \geq 0$, L_1 is regular. (ii) $L_2 = \{w \in \{0,1\}^* \mid n_0(w) = 2n_1(w)\}$. Choose $0^{2p}1^p$ and $y = 01$. Since $xy^iz \in L_2$ for every $i \geq 0$, we have failed to show that L_2 contradicts the lemma. (iii) $L_3 = \{0^n1^n0^n \mid n \geq 0\}$. Choose $0^k1^k0^k$ with $k = \lfloor p/3 \rfloor$. Since $xy^iz \notin L_3$ for every $i \geq 0$ and all possible y, L_3 is not regular.

SOLUTION

(i) Same error as above: pumpability never implies regularity, and y is not ours to choose. L_1 is in fact nonregular (use $s = 0^p10^p1$, $|s| = 2p + 2 \geq p$, exactly as above).

(ii) $y = 01$ is not a legal split: for $s = 0^{2p}1^p \in L_2$ ($|s| = 3p \geq p$), $|xy| \leq p$ places y inside the leading 0^{2p}, so $y = 0^k$, $k \geq 1$. Then $xy^2z = 0^{2p+k}1^p \notin L_2$ (since $2p + k \neq 2p$) — the correct proof that L_2 is not regular. Skye's "failure" comes only from an illegal (and self-chosen) y.

(iii) Two errors. (a) Skye's string has length $3\lfloor p/3 \rfloor \leq p$ — possibly *shorter* than p, and the lemma promises nothing for such a string; the chosen s must satisfy $|s| \geq p$. (b) "$xy^iz \notin L_3$ for every $i \geq 0$" is false — for $i = 1$, $xyz = w \in L_3$; we only need *some* i. The conclusion is true, but the proof must use, e.g., $s = 0^p1^p0^p \in L_3$ ($|s| = 3p \geq p$): then $y = 0^k$, $k \geq 1$, and $xy^0z = 0^{p-k}1^p0^p \notin L_3$ — contradiction.

6 CFG — Lec07 — Context-Free Grammars

Notes: CSE211_Hakim_CFG_Notes.pdf

64. 2023-24 · Q3(c) · 7 marks · ★★★ High · Theory · L18

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2023–2024 • Q3(c) Source N02 — MARKS 7

Why are CFGs called 'context-free'? Explain with examples.

2021–2022 • Q1(e) Source M45 — MARKS 3

Why are context-free grammars called 'context-free'?

SOLUTION

Every production of a CFG has the form $A \to \alpha$ with a *single variable* on the left. Whenever A occurs in a sentential form it may be replaced by α *regardless of the symbols around it* — its context. E.g. with $S \to aSb \mid \varepsilon$, the S in $aaSbb$ is rewritten exactly as the S in S; the surrounding $aa \ldots bb$ is irrelevant.

Contrast with *context-sensitive* grammars, whose rules look like $\alpha A \beta \to \alpha \gamma \beta$: A may be rewritten only when surrounded by α and β. Example rule $bB \to bb$ ("B becomes b only after a b"), used in the grammar for $\{a^n b^n c^n\}$, which no CFG can generate.

Because replacements are independent of context, each variable derives its substring independently — which is exactly why derivations can be drawn as parse *trees*.

65. 2021-22 · Q1(e) · 3 marks · ★★★ High · Theory · L18

Same question — solution already given under #64.

66. 2022-23 · Q5 · 15 marks · ★★★ High · Design · L19

2022–2023 • Q5(i–ii) Source M52 — MARKS (i) 8; (ii) 7

Give CFGs for (i) balanced parentheses over $\{(,)\}$; examples $(()())$, $((()))$, ε are included, but $((())$, $()))$, $)()($ are excluded; (ii) strings with at least two 1's over $\{0,1\}$.

SOLUTION

(i) $S \to (S)S \mid \varepsilon$. A non-empty balanced string starts with "(" whose matching ")" encloses a balanced string, followed by another balanced string. E.g. $(()())$: $S \Rightarrow (S)S \Rightarrow (S) \Rightarrow ((S)S) \Rightarrow (()S) \Rightarrow (()(S)S) \Rightarrow (()()S) \Rightarrow (()())$.

(ii) $S \to A1A1A, \quad A \to 0A \mid 1A \mid \varepsilon$. ($A$ generates any string, so the two explicit 1's guarantee at least two.)

67. 2021-22 · Q2(d) · 15 marks · ★★★ High · Design · L19

2021–2022 • Q2(d) Source M45 — MARKS 15

Design a CFG for $\{a^i b^j c^k \mid i \neq j+k,\ i,j,k \geq 0\}$. Explain the design idea before giving rules; rules alone are insufficient.

SOLUTION

Idea. When $i = j+k$, each c can be matched with an a from the outside in, and then each b with an a: $a^k\,(a^j b^j)\,c^k$. $i \neq j+k$ means either some a's are left unmatched ($i > j+k$) or some b's/c's are left unmatched ($i < j+k$). Generate the matched pairs with recursive rules and force at least one unmatched symbol in the middle.

- $i > j+k$: $a^k\,a^j\,(a^+)\,b^j\,c^k$.
- $i < j+k$, case $i \leq k$: $a^i\,(b^j c^{k-i})\,c^i$ where the middle b^*c^* is non-empty and contains no a's.
- $i < j+k$, case $i > k$: $a^k\,(a^m b^{m+t})\,c^k$ with $t \geq 1$ extra b's.

Grammar (verified):

$$\begin{array}{rcl} S & \to & P \mid Q \\ P & \to & aPc \mid T \qquad T \to aTb \mid A \qquad A \to aA \mid a \qquad (i > j+k) \\ Q & \to & aQc \mid R \mid BC \qquad R \to aRb \mid bB \qquad (i < j+k) \\ B & \to & bB \mid \varepsilon \qquad C \to cC \mid c \end{array}$$

P wraps $a \ldots c$ pairs around T, which wraps $a \ldots b$ pairs around at least one extra a. Q wraps $a \ldots c$ pairs around either R (pairs $a \ldots b$ around at least one extra b) or $BC = b^*c^+$ (extra c's, with any b's). Example: abc ($1 < 2$): $S \Rightarrow Q \Rightarrow aQc \Rightarrow aRc \Rightarrow abBc \Rightarrow abc$.

68. 2021-22 · Q4(c) · 9 marks · ★★★ High · Design · L19

2021–2022 • Q4(c) Source M46 — MARKS 3 + 6 = 9

For $\{a^i b^j c^k \mid i = j \text{ or } j = k;\ i,j,k \geq 0\}$, A. B. Sloppy proposes: (i) $S \to aSb \mid bSc \mid aS \mid Sc \mid \varepsilon$. (ii) $S_0 \to S_1 \mid S_3$; $S_1 \to aS_1bS_2c \mid \varepsilon$; $S_2 \to S_2c \mid \varepsilon$; $S_3 \to aS_4bS_3c \mid \varepsilon$; $S_4 \to aS_4 \mid \varepsilon$. Do these solutions have flaws? Explain clearly.

SOLUTION

(i) Flawed. It over-generates in two ways. Order: $S \Rightarrow bSc \Rightarrow baSbc \Rightarrow babc$, not even of the form $a^*b^*c^*$. Counts: $S \Rightarrow aS \Rightarrow aSc \Rightarrow ac$ has $i = 1, j = 0, k = 1$, so $i \neq j$ and $j \neq k$.
(ii) Flawed. $S_1 \to aS_1bS_2c$ puts c's *between* b's: $S_1 \Rightarrow aS_1bS_2c \Rightarrow a(abc)bc = aabcbc$ (taking the inner $S_1 \Rightarrow abc$, $S_2 \Rightarrow \varepsilon$), not of the form $a^*b^*c^*$. S_3 has the same problem (and S_4 adds a's inside). Neither branch keeps the blocks in order.
Correct grammar (verified):

$$S \to XC \mid AY, \quad X \to aXb \mid \varepsilon, \quad C \to cC \mid \varepsilon, \quad A \to aA \mid \varepsilon, \quad Y \to bYc \mid \varepsilon.$$

XC gives $a^n b^n c^*$ ($i = j$); AY gives $a^* b^n c^n$ ($j = k$).

69. 2020 (Jan term) · Q5 · 15 marks · ★★★ High · Design · L19

January 2020 Term • Q5(a–b) Source M38 — MARKS (a) 7; (b) 8

Give CFGs over $\{0,1\}$ for (a) strings with at least one 0 and at least one 1; (b) strings with equal numbers of 0's and 1's.

SOLUTION

(a) (verified) $S \to A0A1A \mid A1A0A, \quad A \to 0A \mid 1A \mid \varepsilon$.
A generates any binary string; the two explicit symbols guarantee a 0 and a 1, the 0 coming either before ($A0A1A$) or after ($A1A0A$) the 1.
(b) Same grammar as 2019–2020 Q5(ii) above: $S \to 0S1S \mid 1S0S \mid \varepsilon$ (see there for why it generates exactly these strings).

70. 2019-20 · Q5 · 15 marks · ★★★ High · Design · L19

2019–2020 • Q5(i–ii) Source M41 — MARKS (i) 7; (ii) 8

Give CFGs for (i) balanced parentheses over $\{(,)\}$, including $(()())$, $((()))$, and ε, excluding $(())$, $())$, and $)()($; (ii) binary strings with equal numbers of 0's and 1's.

SOLUTION

(i) $S \to (S)S \mid \varepsilon$ (same as 2022–2023 Q5(i) above). A non-empty balanced string starts with "(" whose matching ")" encloses a balanced string, followed by another balanced string.
(ii) (verified)

$$S \to 0S1S \mid 1S0S \mid \varepsilon.$$

Soundness. Every non-ε rule introduces one 0 and one 1, so every generated string has equally many 0's and 1's.
Completeness. Let $w \neq \varepsilon$ have equally many 0's and 1's, and suppose it starts with 0. Scan from left to right until the first point where the prefix has equally many 0's and 1's; that point is reached at a 1. Thus $w = 0\,x\,1\,y$ where x and y both have equally many 0's and 1's (and are shorter than w), so by induction $S \Rightarrow^* x$ and $S \Rightarrow^* y$, and $S \Rightarrow 0S1S \Rightarrow^* 0x1y = w$. If w starts with 1, use $S \to 1S0S$ symmetrically.

71. 2017-18 · Q5 · 15 marks · ★★ Medium · Design · L19

2017–2018 • Q5(a–b) Source M34 — MARKS (a) 7; (b) 8

Provide CFGs over $\{0,1\}$ for (a) equal numbers of 0's and 1's; (b) at least three 1's.

SOLUTION

(a) Same grammar as 2019–2020 Q5(ii) above: $S \to 0S1S \mid 1S0S \mid \varepsilon$.
(b) (verified) $S \to A1A1A1A, \quad A \to 0A \mid 1A \mid \varepsilon$. ($A$ generates any string, so the three explicit 1's guarantee at least three.)

72. 2023-24 · Q5 · 15 marks · ★★★ High · Design · L20

2023–2024 • Q5 Source N03 — MARKS 10 + 5 = 15

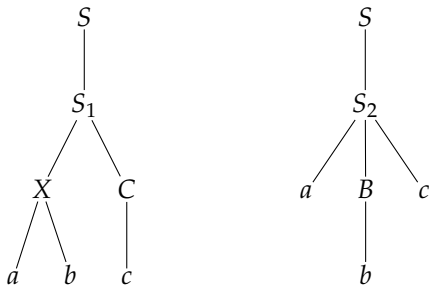
Design a CFG for $\{a^i b^j c^k \mid (i = j \text{ or } i = k),\ i, j, k \geq 1\}$. Show ambiguity using two different parse trees for some string.

SOLUTION

Two sub-grammars, one per condition:

$$\begin{array}{llll} S \to S_1 \mid S_2 & & & \\ S_1 \to XC & X \to aXb \mid ab & C \to cC \mid c & (a^n b^n c^k,\ i = j) \\ S_2 \to aS_2c \mid aBc & B \to bB \mid b & & (a^n b^j c^n,\ i = k) \end{array}$$

A string with $i = j = k$ is produced by both sub-grammars. For abc:



Two distinct parse trees for the same string ⇒ the grammar is ambiguous. (In fact the language is inherently ambiguous: every grammar must generate $a^n b^n c^n$ in two ways for infinitely many n.)

73. 2022-23 · Q6 · 15 marks · ★★★ High · Design · L20

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q6 Source M52 — MARKS 10 + 5 = 15

Give a CFG for $\{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}$. Show ambiguity using two parse trees for some string.

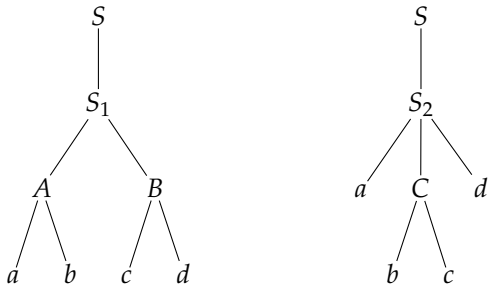
January 2020 Term • Q6 Source M38 — MARKS 10 + 5 = 15

Design a CFG over $\{a, b, c, d\}$ for $\{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}$. Demonstrate ambiguity with two parse trees for some string.

SOLUTION

$$S \to S_1 \mid S_2$$
$$S_1 \to AB \qquad A \to aAb \mid ab \quad B \to cBd \mid cd$$
$$S_2 \to aS_2d \mid aCd \quad C \to bCc \mid bc$$

S_1 generates $a^n b^n c^m d^m$ (two independent matched blocks); S_2 generates $a^n(b^m c^m)d^n$ (nested). Strings with $n = m$, e.g. $abcd$, belong to both:



Hence the grammar is ambiguous. (This language is the textbook example of an *inherently* ambiguous CFL; see 2008–09 Q8(a) below.)

74. 2021-22 · Q4(d) · 10 marks · ★★★ High · Theory · L20

2021–2022 • Q4(d) Source M47 — MARKS 10

For a grammar of an infinite language and string s in it, comment with reasons on ambiguity of both s and the grammar when: (i) s has one parse tree but other strings have multiple trees; (ii) s and every other string have one tree; (iii) s has more than one leftmost derivation.

SOLUTION

A grammar is ambiguous iff *some* string of its language has two or more parse trees (equivalently two leftmost derivations).

(i) s itself is unambiguously derived (one tree), but the **grammar is ambiguous** because some other string has multiple trees. Ambiguity is a property of the grammar witnessed by any single string.

(ii) s is unambiguous and, since *no* string has two trees, the **grammar is unambiguous**.

(iii) Leftmost derivations correspond one-to-one with parse trees, so s has more than one parse tree: s is ambiguously derived and therefore the **grammar is ambiguous**. (The language may or may not be inherently ambiguous — another grammar might be unambiguous.)

75. 2021-22 · Q6 · 15 marks · ★★★ High · Proof · L20

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2021–2022 • Q6 Source M47 — MARKS 6 + 9 = 15

Define ambiguous grammar and inherently ambiguous language. Determine whether this grammar is ambiguous: $S \to AB \mid C$; $A \to aAb \mid ab$; $B \to cB \mid c$; $C \to aCc \mid aDc$; $D \to bD \mid b$.

2019–2020 • Q6 Source M41 — MARKS 5 + 10 = 15

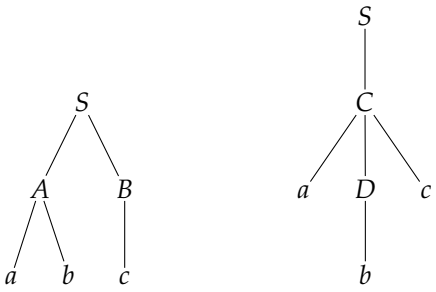
Define ambiguous grammar and inherently ambiguous language. Show ambiguity of the same grammar: $S \to AB \mid C$; $A \to aAb \mid ab$; $B \to cB \mid c$; $C \to aCc \mid aDc$; $D \to bD \mid b$.

SOLUTION

Definitions. A CFG G is *ambiguous* if some $w \in L(G)$ has two (or more) distinct parse trees — equivalently two distinct leftmost derivations. A CFL L is *inherently ambiguous* if *every* CFG generating L is ambiguous.

The grammar. AB generates $a^n b^n c^m$ $(n, m \geq 1)$; C generates $a^n b^m c^n$ $(n, m \geq 1)$. Any $a^n b^n c^n$ is generated both ways. For abc:

$$S \Rightarrow AB \Rightarrow abB \Rightarrow abc \qquad \text{and} \qquad S \Rightarrow C \Rightarrow aDc \Rightarrow abc.$$



Two leftmost derivations / parse trees ⇒ **ambiguous**. (Its language $\{a^n b^n c^m\} \cup \{a^n b^m c^n\}$ is in fact inherently ambiguous.)

76. 2020 (Jan term) · Q6 · 15 marks · ★★★ High · Design · L20

Same question — solution already given under #73.

77. 2019-20 · Q6 · 15 marks · ★★★ High · Proof · L20

Same question — solution already given under #75.

78. 2017-18 · Q6 · 15 marks · ★ Low · Proof · L20

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2017–2018 • Q6(a–b) Source M34 — MARKS (a) 7; (b) 8

(a) Show ambiguity by giving two parse trees for a string in $E \to E + E \mid E \times E \mid (E) \mid a$. (b) Remove the ambiguity.

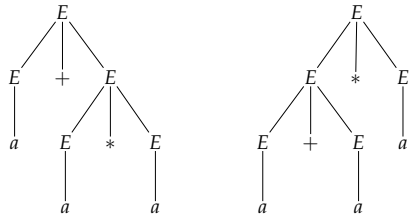
2012–2013 • Q5(a) Source M18 — MARKS 8

For $E \to E + E \mid E * E \mid (E) \mid a$, determine whether the grammar is ambiguous. Explain using $a + a * a$.

SOLUTION

SCOPE *2012–2013 Q5(a) asks only whether the grammar is ambiguous, using $a + a * a$ (first paragraph).*

Ambiguous. $a + a * a$ has two parse trees (write $\times$ or $*$ alike):

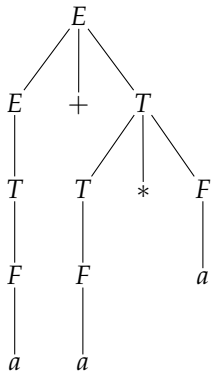


Leftmost derivations: $E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E \Rightarrow a + a * E \Rightarrow a + a * a$ and $E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E \Rightarrow a + a * E \Rightarrow a + a * a$. The first tree means $a + (a * a)$, the second $(a + a) * a$ — different meanings.

Sources of ambiguity: no precedence between $+$ and $*$, and no associativity ($a + a + a$ also has two trees).
Removing it. Introduce one variable per precedence level, and make each operator left-associative by putting the recursion on the left:

$$\begin{aligned} E &\to E + T \mid T \\ T &\to T * F \mid F \\ F &\to (E) \mid a \end{aligned}$$

Now $a + a * a$ has the single tree corresponding to $a + (a * a)$:



A $+$ can only appear at the E level, above any $*$, so $*$ binds tighter; and in $E \to E + T$ the right operand cannot contain an unparenthesised $+$, so $a + a + a$ groups as $(a + a) + a$.